
Guía paso a paso para aprender React

Desde la instalación de Node.js hasta tu primera aplicación

Node.js · npm · Vite · React 19 · Jasmine ·
Karma

JSX, componentes, props, estilos, estado, eventos, listas, formularios,
efectos y pruebas unitarias con Jasmine y Karma

Contenido

1	Antes de empezar	4
1.1	Qué es el DOM	4
1.2	Imperativo y declarativo	4
1.3	Qué es un componente	5
2	El JavaScript que necesitas	5
2.1	const y let	5
2.2	Template literals	5
2.3	Funciones flecha	5
2.4	Desestructuración	6
2.5	Spread (...)	6
2.6	Métodos de arreglos	6
2.7	Condiciones cortas	7
2.8	Módulos: import y export	7
2.9	Promesas y async/await	7
3	Instalación de Node.js	8
3.1	Windows	8
3.2	macOS	8
3.3	Linux (Ubuntu / Debian)	8
3.4	Opción recomendada: nvm	8
3.5	Verificar la instalación	9
4	Preparar el editor	9
5	Crear el primer proyecto con Vite	10
5.1	Generar el proyecto	10
5.2	Instalar dependencias y ejecutar	10
5.3	Proyecto de acompañamiento	10
6	Estructura del proyecto	11
6.1	Los scripts de package.json	11
6.2	Dejar el proyecto limpio	11
7	Cómo arranca la aplicación: los componentes del render	12
7.1	index.html y el div raíz	12

7.2	<code>main.jsx</code> , línea por línea	13
7.3	El árbol de componentes	14
7.4	Las tres fases de cada render	14
8	JSX: HTML dentro de JavaScript	14
8.1	Reglas básicas	14
8.2	Estilos	15
8.3	Qué se puede poner entre llaves	15
8.4	Más ejemplos	16
9	Componentes	17
9.1	Más ejemplos	17
9.2	Cómo dividir en componentes	18
10	Estilos: CSS en React	18
10.1	Cuatro formas de aplicar estilos	19
10.2	Un detalle importante: el CSS importado es global	19
10.3	CSS global: variables y reglas base	19
10.4	CSS por componente	21
10.5	CSS Modules	22
10.6	Clases dinámicas	23
10.7	Estilos en línea	23
10.8	Cómo quedan los archivos	23
10.9	Errores frecuentes con estilos	24
11	Props: pasar datos a un componente	24
11.1	La prop especial <code>children</code>	25
11.2	Tipos de valores en props	25
11.3	Más ejemplos	25
12	Estado con <code>useState</code>	27
12.1	Tres reglas que evitan errores	27
12.2	¿Props o estado?	28
12.3	Más ejemplos	28
13	Eventos	30
13.1	El objeto evento	30
13.2	Eventos más usados	31
13.3	Más ejemplos	31

14 Renderizado condicional	32
14.1 Más ejemplos	32
15 Listas y la prop key	34
15.1 Por qué React necesita key	34
15.2 Más ejemplos	35
16 Formularios controlados	36
16.1 Cada tipo de campo	37
16.2 Más ejemplos	37
17 Efectos con useEffect	38
17.1 Ejemplo: consumir una API	39
17.2 Cuándo usar useEffect y cuándo no	40
17.3 Más ejemplos	40
18 Cuándo se vuelve a renderizar un componente	42
18.1 Experimento: pestaña “Renderizado”	42
18.2 El estado es una foto de cada render	43
19 Proyecto integrador: lista de tareas	44
20 Pruebas unitarias con Jasmine y Karma	46
20.1 Qué hace cada herramienta	46
20.2 Instalación y configuración	47
20.3 Anatomía de una prueba	48
20.4 Primer ejemplo	48
20.5 Comparadores de Jasmine	50
20.6 Preparar datos con beforeEach	50
20.7 Probar los casos límite	51
20.8 Separar la lógica de la interfaz	52
20.9 Espías	52
20.10 Pruebas asíncronas	53
20.11 Probar un componente	53
20.12 Ejecutar las pruebas	55
20.13 Errores frecuentes al probar	55
20.14 Buenas prácticas	55
20.15 La alternativa actual: Vitest	56

21 Generar la versión de producción	56
22 Errores frecuentes	57
23 Resumen de comandos	57
24 Glosario	58
25 Próximos pasos	58

1 Antes de empezar

React es una biblioteca de JavaScript para construir interfaces de usuario a partir de piezas reutilizables llamadas **componentes**. Cada componente es una función que devuelve lo que se debe mostrar en pantalla, y React se encarga de actualizar la página cuando los datos cambian.

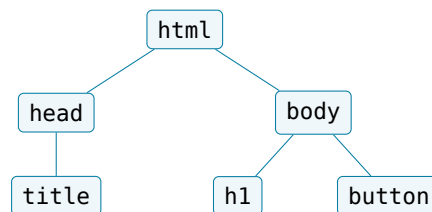
Para trabajar con React en tu computador necesitas cuatro cosas:

Herramienta	Para qué sirve
Node.js	Entorno que ejecuta JavaScript fuera del navegador. Lo usan las herramientas de desarrollo.
npm	Gestor de paquetes que viene incluido con Node.js. Instala React y sus dependencias.
Vite	Herramienta que crea el proyecto, levanta un servidor de desarrollo y genera la versión final.
Editor	Se recomienda Visual Studio Code.

Conocimientos previos recomendados: HTML, CSS y JavaScript básico (variables, funciones, arreglos, objetos, `map` y `filter`, funciones flecha y desestructuración). La sección 2 repasa todo lo que necesitas de JavaScript moderno.

1.1 Qué es el DOM

Cuando el navegador lee un archivo HTML, construye en memoria un árbol de objetos que representa la página: el **DOM** (*Document Object Model*). Cada etiqueta es un nodo de ese árbol, y JavaScript puede leerlo y modificarlo. Todo lo que ves en pantalla sale del DOM.



Modificar el DOM a mano es posible, pero se vuelve difícil de mantener cuando la página crece. React resuelve eso.

1.2 Imperativo y declarativo

Compara un contador escrito con JavaScript puro y con React.

JavaScript puro (imperativo). Tú das cada instrucción para cambiar la pantalla:

```
1 let cuenta = 0
2 const boton = document.querySelector('#boton')
3 const texto = document.querySelector('#texto')
4
5 boton.addEventListener('click', () => {
6   cuenta = cuenta + 1
7   texto.textContent = `Clicks: ${cuenta}` // tu actualizas el DOM
8 })
```

React (declarativo). Describes cómo debe verse la pantalla para cada valor de los datos, y React hace los cambios:

```
1 function Contador() {  
2   const [cuenta, setCuenta] = useState(0)  
3   return <button onClick={() => setCuenta(cuenta + 1)}>Clicks: {cuenta}</button>  
4 }
```

En el segundo caso no hay ninguna línea que modifique el DOM. Solo cambias el dato (*cuenta*) y la pantalla se ajusta. Esa es la idea central de React: **la interfaz es una función de los datos**.

1.3 Qué es un componente

Mira cualquier sitio web y divídelo en rectángulos: barra de navegación, tarjeta de producto, botón, pie de página. Cada rectángulo que tiene sentido por sí solo puede ser un componente. Un componente reúne en un mismo lugar la estructura (JSX), el aspecto (CSS) y el comportamiento (JavaScript) de esa parte, y se puede usar tantas veces como quieras.

2 El JavaScript que necesitas

React es JavaScript. La mayoría de las dificultades al empezar vienen de sintaxis moderna del lenguaje y no de React. Esta sección repasa lo que vas a usar todo el tiempo.

Nota

Todos los ejemplos de esta sección están en `repaso-js/repaso.js` del proyecto de acompañamiento. Ejecútalos con `npm run repaso` y verás los resultados en la terminal. También puedes probar cada línea en la consola del navegador (F12).

2.1 `const` y `let`

```
1 const curso = 'React' // no se puede reasignar  
2 let inscritos = 30    // se puede reasignar  
3 inscritos = inscritos + 5
```

Usa `const` por defecto y `let` solo cuando el valor deba cambiar. Evita `var`. En React casi todo es `const`, incluso el estado: `const [cuenta, setCuenta]`. El valor no se reasigna; React entrega uno nuevo en cada render.

2.2 Template literals

Texto entre comillas invertidas (```) que permite insertar variables con `${ }`:

```
console.log(`El curso ${curso} tiene ${inscritos} inscritos`)
```

En React se usan para armar clases CSS o URLs: `className={`boton boton-${variante}`}`.

2.3 Funciones flecha

```

1 function sumarClasica(a, b) {
2   return a + b
3 }
4 const sumar = (a, b) => a + b           // una expresion: retorno implicito
5 const doble = (n) => n * 2
6 const crearUsuario = (nombre) => ({ nombre, activo: true }) // objeto entre ()

```

Si el cuerpo es una sola expresión, se omiten las llaves y el `return`. Para devolver un objeto de forma implícita hay que envolverlo en paréntesis, porque las llaves solas se leerían como bloque. En React las verás en eventos (`onClick={() => ...}`) y dentro de `map`.

2.4 Desestructuración

Extrae valores de objetos y arreglos en variables:

```

1 const alumno = { nombre: 'Pedro', edad: 22, carrera: 'Informatica' }
2 const { nombre, edad } = alumno
3 const { carrera: programa, sede = 'Vina del Mar' } = alumno // renombrar y por defecto
4
5 const colores = ['rojo', 'verde', 'azul']
6 const [primero, segundo] = colores

```

Con objetos importan los **nombres**; con arreglos importa la **posición**. Las props se reciben desestructurando un objeto (`function Tarjeta({ titulo })`) y `useState` devuelve un arreglo de dos posiciones, por eso puedes llamarlas como quieras.

2.5 Spread (...)

Copia el contenido de un arreglo u objeto dentro de otro nuevo:

```

1 const masColores = [...colores, 'negro'] // copia y agrega
2 const alumnoMayor = { ...alumno, edad: 23 } // copia y reemplaza edad
3 // alumno sigue teniendo edad 22

```

Es la herramienta principal para actualizar estado sin modificar el original.

2.6 Métodos de arreglos

Ninguno de estos métodos modifica el arreglo original.

```

1 const productos = [
2   { id: 1, nombre: 'Teclado', precio: 25990, stock: 4 },
3   { id: 2, nombre: 'Mouse', precio: 12990, stock: 0 },
4   { id: 3, nombre: 'Monitor', precio: 149990, stock: 2 },
5 ]
6 productos.map((p) => p.nombre)           // ['Teclado', 'Mouse', 'Monitor']
7 productos.filter((p) => p.stock > 0)     // Teclado y Monitor
8 productos.find((p) => p.id === 3)        // el objeto del Monitor
9 productos.some((p) => p.stock === 0)     // true
10 productos.reduce((total, p) => total + p.precio, 0) // 188970

```

Método	Devuelve	Uso típico en React
map	Arreglo nuevo del mismo largo, transformado.	Convertir datos en elementos JSX.
filter	Arreglo nuevo solo con los que cumplen.	Buscar, eliminar un elemento del estado.
find	El primer elemento que cumple, o undefined.	Obtener el elemento seleccionado.
some	true o false.	Saber si mostrar un aviso.
reduce	Un solo valor acumulado.	Totales y sumas.

Atención

sort, push, pop, splice y reverse sí modifican el arreglo original. Con estado, aplícalos sobre una copia: [...lista].sort().

2.7 Condiciones cortas

```

1 nota >= 4 ? 'Aprobado' : 'Reprobado' // ternario: una cosa u otra
2 nota > 6 && 'Con distincion'         // && : el valor si se cumple, false si no
3 apodo ?? 'Sin apodo'                // ?? : valor de reemplazo si es null o undefined
4 config.idioma?.codigo                // ?. : undefined en vez de error si no existe

```

Dentro de JSX no se puede escribir un `if`, así que estas formas son las que se usan en el `return`. El `?.` es muy útil con datos que llegan de una API y todavía no están cargados.

2.8 Módulos: `import` y `export`

repaso-js/utilidades.js

```

1 export const IVA = 0.19 // exportacion nombrada
2 export function conIva(precio) { return Math.round(precio * (1 + IVA)) }
3 export default function formatearPesos(valor) { // exportacion por defecto
4   return '$' + valor.toLocaleString('es-CL')
5 }

```

```

1 import formatearPesos, { IVA, conIva } from './utilidades.js'

```

Un archivo tiene como máximo un `export default`, que se importa sin llaves y con el nombre que quieras. Las exportaciones nombradas se importan entre llaves con su nombre exacto. Por eso `import { useState } from 'react'` lleva llaves.

2.9 Promesas y `async/await`

Algunas operaciones tardan, como pedir datos a un servidor. Una **promesa** representa un resultado que llegará después. `await` espera ese resultado sin bloquear la página, y solo se puede usar dentro de una función `async`:

```

1 async function cargarUsuarios() {
2   try {
3     const res = await fetch('https://jsonplaceholder.typicode.com/users')
4     const datos = await res.json()
5     console.log(datos)
6   } catch (error) {

```

```
7 console.log('Algo fallo:', error.message)
8   }
9 }
```

Es otra forma de escribir las cadenas `.then().catch()`. Las dos hacen lo mismo; `async/await` suele leerse mejor.

Ejercicio de JavaScript

Con el arreglo `productos`, obtén en una sola línea los nombres de los productos con stock, en mayúsculas. Luego calcula el valor total del inventario (precio por stock de cada producto).

3 Instalación de Node.js

Instala siempre la versión **LTS** (*Long Term Support*), que es la versión estable recomendada para trabajar. Vite exige Node.js 20.19 o superior (o 22.12 en adelante), así que cualquier LTS vigente sirve.

3.1 Windows

Opción A: instalador oficial.

1. Entra a <https://nodejs.org> y descarga el instalador LTS (.msi).
2. Ejecútalo y acepta las opciones por defecto. Deja marcada la casilla que agrega Node al PATH.
3. Cierra y vuelve a abrir la terminal (PowerShell o CMD) para que reconozca los comandos nuevos.

Opción B: desde la terminal con winget.

```
winget install OpenJS.NodeJS.LTS
```

3.2 macOS

Puedes usar el instalador .pkg de <https://nodejs.org> o Homebrew:

```
brew install node
```

3.3 Linux (Ubuntu / Debian)

Los repositorios de la distribución suelen traer versiones antiguas. Lo más limpio es usar **nvm** (ver sección 3.4).

3.4 Opción recomendada: nvm

nvm (*Node Version Manager*) permite tener varias versiones de Node y cambiar entre ellas. Es útil cuando un proyecto antiguo pide otra versión.

En macOS y Linux:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/master/install.sh | bash
# cierra y abre la terminal
nvm install --lts
nvm use --lts
```

En Windows se usa un proyecto distinto, **nvm-windows** (<https://github.com/coreybutler/nvm-windows>). Tras instalarlo:

```
nvm install lts
nvm use lts
```

3.5 Verificar la instalación

```
node -v
npm -v
```

Ambos comandos deben responder con un número de versión, por ejemplo `v22.x.x` y `10.x.x`.

Atención

Si aparece “*node no se reconoce como un comando*”, cierra todas las terminales (y VS Code) y ábrealas de nuevo. Si el problema sigue, reinicia el equipo o revisa que la carpeta de Node esté en la variable PATH.

Atención

En Windows, PowerShell puede bloquear npm con un error de *ejecución de scripts deshabilitada*. Abre PowerShell y ejecuta: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

4 Preparar el editor

1. Descarga Visual Studio Code desde <https://code.visualstudio.com>.
2. Instala estas extensiones desde el panel de extensiones (Ctrl+Shift+X):
 - **ES7+ React/Redux/React-Native snippets**: atajos como `rafce` que generan un componente completo.
 - **Oxc**: muestra en el editor los avisos de `oxlint`, el revisor de código que trae la plantilla de Vite.
 - **Prettier**: formatea el código al guardar.
3. En el navegador (Chrome o Firefox) instala **React Developer Tools**. Agrega las pestañas *Components* y *Profiler* a las herramientas de desarrollo para inspeccionar componentes, props y estado.

5 Crear el primer proyecto con Vite

Nota

Durante años se usó `create-react-app`. Hoy está descontinuado y la documentación oficial de React ya no lo recomienda. Para aprender, Vite es la opción más simple y rápida.

5.1 Generar el proyecto

Abre una terminal en la carpeta donde guardas tus proyectos y ejecuta:

```
npm create vite@latest mi-primer-app
```

El asistente hará dos preguntas. Responde así:

- **Select a framework:** React
- **Select a variant:** JavaScript (más adelante puedes probar TypeScript)

También puedes indicar la plantilla directamente y saltarte las preguntas:

```
npm create vite@latest mi-primer-app -- --template react
```

5.2 Instalar dependencias y ejecutar

```
cd mi-primer-app
npm install
npm run dev
```

La terminal mostrará una dirección como `http://localhost:5173`. Ábrela en el navegador y verás la página de ejemplo con un contador.

Nota

El servidor de desarrollo tiene *recarga en caliente* (HMR): cada vez que guardas un archivo, el navegador se actualiza solo. Para detener el servidor presiona `Ctrl+C` en la terminal.

Para abrir el proyecto en VS Code desde la terminal:

```
code .
```

5.3 Proyecto de acompañamiento

Junto a esta guía se entrega `guia-react.zip`, con todos los ejemplos funcionando. Cada pestaña de la aplicación corresponde a un capítulo y cada ejemplo muestra la ruta del archivo que lo contiene, para que puedas abrirlo, modificarlo y ver el cambio al instante. Para usarlo:

```
# descomprime el zip y entra a la carpeta
cd guia-react
npm install          # la carpeta node_modules no viene en el zip
npm run dev
```

```

karma.conf.cjs      configuracion de las pruebas
repaso-js/          repaso de JavaScript (npm run repaso)
docs/               esta guia en PDF y LaTeX
src/
|-- main.jsx         punto de entrada comentado
|-- App.jsx          navegacion entre secciones
|-- index.css        variables y reglas base
|-- styles/utilidades.css  clases compartidas
|-- logica/          funciones puras y sus archivos .spec.js
`-- components/
    |-- Ejemplo.jsx   marco con titulo y archivo de cada ejemplo
    |-- basicos/       ejemplos principales de cada capitulo
    |                  cada componente con estilos trae su .css al lado
    |-- jsx/ props/ estado/ eventos/ condicional/
    |-- listas/ formularios/ efectos/   ejemplos adicionales
    |-- render/DemoRender.jsx experimento de renderizado
    `-- tareas/        proyecto integrador

```

6 Estructura del proyecto

```

mi-primer-app/
|-- node_modules/    dependencias instaladas (no se edita ni se sube a Git)
|-- public/          archivos estaticos servidos tal cual
|-- src/
|   |-- assets/       imagenes y recursos importados desde el codigo
|   |-- App.css
|   |-- App.jsx       componente principal
|   |-- index.css     estilos globales
|   `-- main.jsx      punto de entrada: monta React en la pagina
|-- index.html        unica pagina HTML; contiene <div id="root">
|-- package.json      nombre, scripts y dependencias
|-- vite.config.js    configuracion de Vite
`-- .oxlintrc.json    reglas de oxlint (revisor de codigo)

```

El recorrido completo desde `index.html` hasta tu componente se explica en la sección 7.

6.1 Los scripts de `package.json`

Comando	Qué hace
<code>npm run dev</code>	Levanta el servidor de desarrollo.
<code>npm run build</code>	Genera la versión optimizada para producción en la carpeta <code>dist/</code> .
<code>npm run preview</code>	Sirve localmente el contenido de <code>dist/</code> para probarlo.
<code>npm run lint</code>	Revisa el código con <code>oxlint</code> .
<code>npm test</code>	Ejecuta las pruebas unitarias con Karma y Jasmine.
<code>npm run repaso</code>	Corre el repaso de JavaScript en la terminal.

6.2 Dejar el proyecto limpio

Antes de escribir tu propio código, reemplaza el contenido de `src/App.jsx` por:

src/App.jsx

```

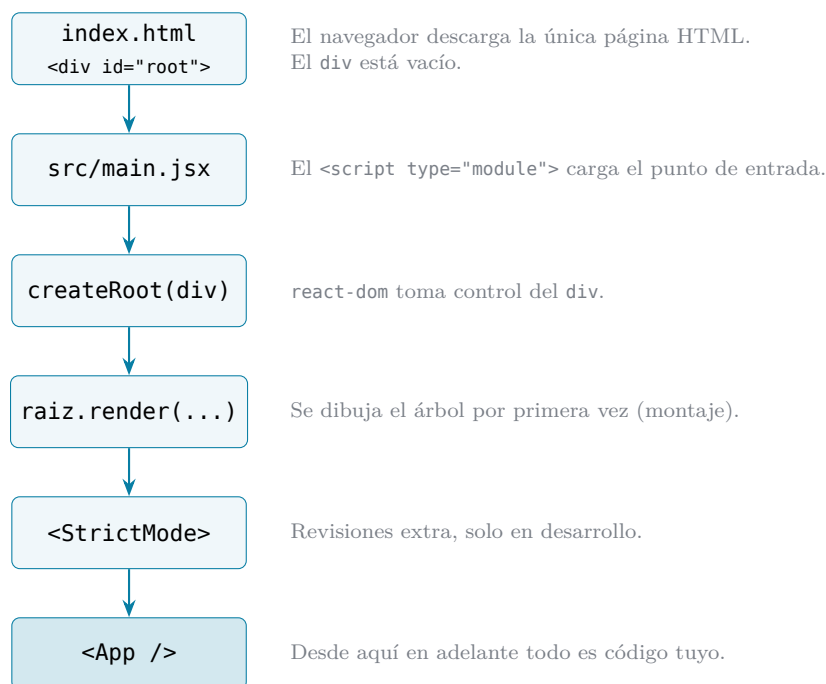
1  function App() {
2    return (
3      <main>
4        <h1>Hola, React</h1>
5        <p>Mi primera aplicacion.</p>
6      </main>
7    )
8  }
9
10 export default App

```

Borra también el contenido de `src/App.css` y `src/index.css` para partir sin estilos heredados.

7 Cómo arranca la aplicación: los componentes del render

Entre abrir `localhost:5173` y ver tu componente en pantalla intervienen varias piezas. Conocerlas ayuda a entender los errores de arranque y qué hace cada línea de `main.jsx`.



7.1 index.html y el div raíz

Es la única página HTML de la aplicación. Su `body` contiene dos líneas:

```

<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>

```

Si abres “ver código fuente” en el navegador verás ese `div` vacío. Todo el contenido lo inserta JavaScript después. Por eso a este tipo de aplicación se le llama *SPA (Single Page Application)*: hay una sola página y React cambia su contenido.

7.2 main.jsx, línea por línea

src/main.jsx

```
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.jsx'
5
6 const contenedor = document.getElementById('root')
7 const raiz = createRoot(contenedor)
8
9 raiz.render(
10   <StrictMode>
11     <App />
12   </StrictMode>,
13 )
```

La plantilla de Vite escribe las líneas 6 a 9 en una sola expresión encadenada. Aquí están separadas para ver cada paso.

Líneas 1 y 2: dos paquetes distintos.

react define qué es un componente, JSX y los hooks. react-dom sabe traducir eso a elementos del navegador (el DOM). Esta separación existe porque React también dibuja en otros destinos: React Native usa el mismo react con otro paquete de dibujo.

Línea 3: import './index.css'.

Importar un CSS desde JavaScript no es estándar del lenguaje. Vite lo intercepta e inyecta los estilos en la página. Los estilos quedan globales.

Línea 4: import App.

Trae el componente raíz de tu aplicación.

Línea 6: document.getElementById('root').

Es DOM puro, no React. Busca el div de index.html. Si cambias el id en un archivo y no en el otro, aparece el error *Target container is not a DOM element*.

Línea 7: createRoot(contenedor).

Crea una **raíz de React** sobre ese div. A partir de este momento React administra todo lo que ocurre dentro de él, y no debes modificar ese contenido con innerHTML u otros métodos del DOM. Se llama una sola vez en toda la aplicación.

Línea 9: raiz.render(...).

Recibe un elemento JSX y lo dibuja por primera vez. A este primer dibujo se le llama **montaje**. Después no vuelves a llamar a render: las actualizaciones posteriores las disparan los cambios de estado dentro de los componentes.

Línea 10: <StrictMode>.

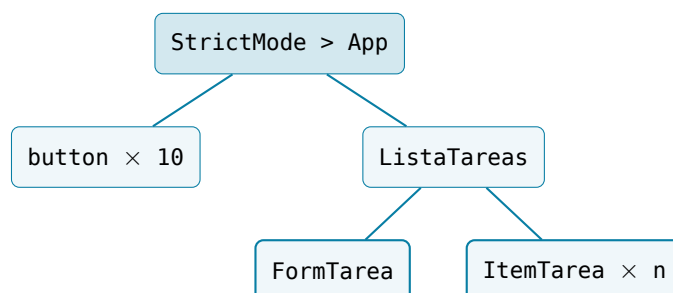
Es un componente que no produce nada visible. Solo actúa en desarrollo y desaparece en npm run build. Hace dos cosas: ejecuta cada componente dos veces para detectar código impuro, y monta, desmonta y vuelve a montar los efectos para revelar limpiezas olvidadas. Por eso ves console.log duplicados o dos peticiones fetch en la pestaña Red.

Línea 11: <App />.

Es la raíz de tu árbol de componentes. Todo lo que construyas cuelga de aquí.

7.3 El árbol de componentes

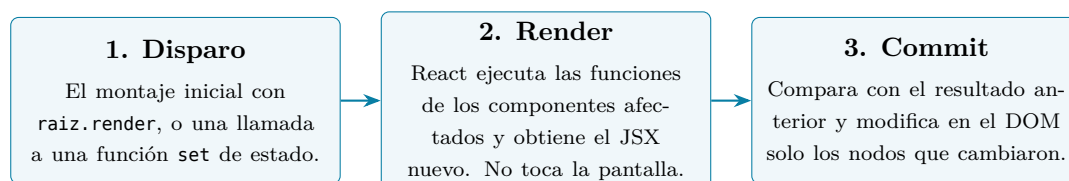
Cada componente puede usar otros dentro de su JSX, así que la aplicación forma un árbol. Este es el del proyecto de acompañamiento cuando está activa la pestaña de tareas:



Los datos bajan por el árbol mediante props (de `ListaTareas` a `ItemTarea`). Los avisos suben mediante funciones recibidas como props (`onEliminar`, `onAgregar`).

7.4 Las tres fases de cada render

En React, **renderizar** significa llamar a la función del componente para obtener su JSX. Que un componente se renderice no implica que la pantalla cambie. Cada actualización pasa por tres fases:



Una forma de verlo: la fase de render dibuja un plano nuevo de la casa y la fase de commit ejecuta en obra solo las diferencias con el plano anterior. Si nada cambió, la obra no se toca.

Esta es la razón por la que la función de un componente debe ser **pura**: con las mismas props y el mismo estado debe devolver el mismo JSX, sin modificar variables externas, sin hacer peticiones y sin tocar el DOM. Lo que tenga efectos externos va en manejadores de eventos o en `useEffect`.

Nota

En la sección 18 se explica qué provoca que un componente se vuelva a renderizar, una vez que conozcas el estado.

8 JSX: HTML dentro de JavaScript

JSX es una sintaxis que permite escribir estructuras parecidas a HTML dentro de JavaScript. Vite la traduce a llamadas de JavaScript normales.

8.1 Reglas básicas

1. **Un solo elemento raíz.** Si necesitas devolver varios elementos, envuélvelos en un `div` o en un fragmento vacío `<>...</>`.

2. **Todas las etiquetas se cierran**, incluso las que en HTML no lo requieren: `<img />`, `<br />`, `<input />`.
3. **Atributos en camelCase**: `class` pasa a ser `className`, `for` pasa a ser `htmlFor`, `onclick` pasa a ser `onClick`.
4. **Las llaves { } insertan JavaScript**: variables, operaciones o llamadas a funciones.

```

1 function Perfil() {
2   const nombre = 'Ana'
3   const edad = 21
4   const foto = 'https://i.pravatar.cc/100'
5
6   return (
7     <>
8       <img src={foto} alt={nombre} className="avatar" />
9       <h2>{nombre.toUpperCase()}</h2>
10      <p>El proximo anio tendra {edad + 1} anios.</p>
11    </>
12  )
13 }

```

8.2 Estilos

La forma habitual es escribir CSS normal en un archivo e importarlo desde el componente:

```

1 import './Perfil.css'
2 // ...
3 <div className="tarjeta">...</div>

```

También se pueden poner estilos en línea con el atributo `style`, que recibe un **objeto** con las propiedades en camelCase. Por eso aparecen dobles llaves: las externas abren JavaScript y las internas son las del objeto.

```

1 <p style={{ color: 'white', backgroundColor: '#087EA4', padding: 8 }}>
2   Texto destacado
3 </p>

```

La sección 10 trata el tema completo: dónde poner cada archivo, variables CSS, CSS Modules y clases dinámicas.

8.3 Qué se puede poner entre llaves

Entre llaves va cualquier **expresión**: algo que produce un valor. Variables, operaciones, llamadas a funciones, ternarios y `map` son expresiones. Las **sentencias** como `if`, `for` o `const x = 1` no lo son y no se pueden escribir dentro del JSX. Se escriben antes del `return`.

Válido	No válido
<code>{nombre}</code>	<code>{if (activo) { ... }}</code>
<code>{precio * 1.19}</code>	<code>{for (let i = 0; ...)}</code>
<code>{activo ? 'Sí' : 'No'}</code>	<code>{const x = 5}</code>
<code>{lista.map(...)}</code>	<code>{alumno} si es un objeto</code>

Un objeto no se puede mostrar directamente: React no sabe cómo convertirlo en texto y lanza el error *Objects are not valid as a React child*. Muestra sus propiedades (`{alumno.nombre}`) o conviértelo con `JSON.stringify`. Los valores `true`, `false`, `null` y `undefined` no muestran nada.

8.4 Más ejemplos

Datos, cálculos y JSX en variables. Antes del `return` puedes hacer cualquier cálculo. Aquí se obtienen las iniciales con métodos de texto y arreglos, y se guarda un fragmento de JSX en una variable para usarlo después.

src/components/jsx/TarjetaPresentacion.jsx

```
1 // Datos fuera del componente: no cambian, así que no necesitan estado.
2 const persona = {
3   nombre: 'Valentina Rojas',
4   carrera: 'Ingeniería en Informática',
5   semestre: 2,
6   intereses: ['React', 'Bases de datos', 'Diseño'],
7 }
8
9 function TarjetaPresentacion() {
10   // Cálculos normales de JavaScript antes del return
11   const iniciales = persona.nombre
12     .split(' ')
13     .map((palabra) => palabra[0])
14     .join('')
15   const esPrimerAnio = persona.semestre <= 2
16
17   // Una variable también puede guardar JSX
18   const etiqueta = esPrimerAnio ? <span className="badge">Primer año</span> : null
19
20   return (
21     <div className="caja">
22       /* Así se escribe un comentario dentro de JSX */
23       <div className="avatar-texto">{iniciales}</div>
24       <h3>
25         {persona.nombre} {etiqueta}
26       </h3>
27       <p>
28         {persona.carrera}, semestre {persona.semestre}
29       </p>
30       <p>Intereses: {persona.intereses.join(', ')}</p>
31       <p>Cantidad de intereses: {persona.intereses.length}</p>
32     </div>
33   )
34 }
35
36 export default TarjetaPresentacion
```

Observa que `persona` está fuera de la función. Los datos que no cambian no necesitan estado ni tienen que redefinirse en cada render.

9 Componentes

Un componente es una función cuyo nombre empieza con **mayúscula** y que devuelve JSX. La mayúscula es obligatoria: así React distingue `<Boton />` (tu componente) de `<button>` (etiqueta HTML).

Crea la carpeta `src/components` y dentro el archivo `Saludo.jsx`:

`src/components/Saludo.jsx`

```
1 function Saludo() {  
2   return <h2>Bienvenido al curso</h2>  
3 }  
4  
5 export default Saludo
```

Úsalo en `App.jsx`:

`src/App.jsx`

```
1 import Saludo from './components/Saludo'  
2  
3 function App() {  
4   return (  
5     <main>  
6       <h1>Hola, React</h1>  
7       <Saludo />  
8       <Saludo />  
9     </main>  
10  )  
11 }  
12  
13 export default App
```

Nota

Con la extensión de snippets instalada, escribe `rafce` en un archivo vacío y presiona `Tab`. Se genera el esqueleto del componente con el nombre del archivo.

9.1 Más ejemplos

Composición. Construir una interfaz consiste en combinar componentes pequeños. Este archivo define tres componentes y exporta dos de ellos por nombre, además del componente por defecto.

`src/components/jsx/Composicion.jsx`

```
1 // Un archivo puede tener varios componentes.  
2 // "export" sin default permite importarlos por nombre: import { Encabezado } from './  
3   Composicion'  
4  
5 export function Encabezado({ titulo }) {  
6   return (  
7     <header className="encabezado">  
8       <strong>{titulo}</strong>  
9     </header>  
10  )  
11 }  
12  
13 export function PiePagina() {
```

```

13   const anio = new Date().getFullYear()
14   return <footer className="pie">© {anio} Mi sitio</footer>
15 }
16
17 // Layout arma la estructura y deja un espacio (children) para el contenido.
18 function Layout({ titulo, children }) {
19   return (
20     <div className="layout">
21       <Encabezado titulo={titulo} />
22       <div className="contenido">{children}</div>
23       <PiePagina />
24     </div>
25   )
26 }
27
28 function Composicion() {
29   return (
30     <Layout titulo="Mi tienda">
31       <p>Este contenido lo decide quien usa Layout.</p>
32       <p>El encabezado y el pie se repiten sin volver a escribirlos.</p>
33     </Layout>
34   )
35 }
36
37 export default Composicion

```

Layout no sabe qué contenido va a mostrar: lo recibe en children. Así puedes tener una misma estructura de página con contenidos distintos. Desde otro archivo se importaría así:

```
import Composicion, { Encabezado, PiePagina } from './components/jsx/Composicion'
```

9.2 Cómo dividir en componentes

Algunas pistas para decidir cuándo crear un componente nuevo:

- Una parte de la interfaz se repite (tarjetas, filas de una tabla, botones con el mismo estilo).
- Un componente supera las 80 o 100 líneas y cuesta leerlo.
- Una parte tiene su propio estado que al resto no le interesa.

Un archivo por componente es la convención más común. El nombre del archivo coincide con el del componente.

10 Estilos: CSS en React

React no trae su propio sistema de estilos: se usa CSS normal. Lo que cambia es **dónde** vive ese CSS y **cómo** llega a la página.

10.1 Cuatro formas de aplicar estilos

Forma	Cómo se escribe	Cuándo conviene
CSS global	Un archivo importado en <code>main.jsx</code> .	Variables, reglas base y clases compartidas.
CSS por componente	Un <code>.css</code> junto al <code>.jsx</code> , importado por él.	Lo más común. Mantiene cerca el componente y su estilo.
CSS Modules	Archivo <code>.module.css</code> importado como objeto.	Cuando quieres nombres de clase que no puedan chocar.
Estilos en línea	Atributo <code>style</code> con un objeto.	Valores calculados en tiempo de ejecución.

Además existen herramientas externas como Tailwind CSS, que usa clases utilitarias, o styled-components, que escribe el CSS dentro del JavaScript. No las necesitas para empezar.

10.2 Un detalle importante: el CSS importado es global

Cuando escribes `import './Boton.css'` dentro de `Boton.jsx`, Vite inserta ese archivo en la página completa. El `import` sirve para **organizar** el proyecto y para que Vite sepa qué incluir, pero la clase `.boton` queda disponible en toda la aplicación. Si otro archivo define también `.boton`, la última regla cargada gana.

Dos maneras de evitar choques de nombres:

1. **Prefijar las clases** con el nombre del componente: `.tarjeta`, `.tarjeta-titulo`, `.tarjeta-precio`. Es la convención más usada y no requiere nada especial.
2. **CSS Modules**, donde el navegador nunca ve tus nombres originales.

10.3 CSS global: variables y reglas base

Las **variables CSS** se definen una vez en `:root` y se usan en todos los archivos con `var(--nombre)`. Cambiar un color del tema pasa a ser editar una línea.

src/index.css

```

1  /* Estilos globales: se cargan una vez desde main.jsx y valen para toda la app. */
2
3  /* Variables CSS: se definen una vez y se reutilizan en todos los archivos */
4  :root {
5      --color-principal: #087ea4;
6      --color-principal-suave: #e6f4f9;
7      --color-texto: #23272f;
8      --color-tenue: #7a7f87;
9      --color-borde: #e3e6ea;
10     --color-fondo: #f6f7f9;
11     --color-alerta: #b45309;
12     --color-ok: #15803d;
13     --color-peligro: #c0306f;
14     --radio: 8px;
15     --espacio: 12px;
16
17     font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
18     color: var(--color-texto);

```

```

19   background: var(--color-fondo);
20 }
21
22 * {
23   box-sizing: border-box;
24 }
25
26 body {
27   margin: 0;
28 }
29
30 /* Estilos base de los elementos del formulario, iguales en toda la app */
31 button {
32   font: inherit;
33   padding: 6px 12px;
34   margin: 4px 4px 4px 0;
35   border: 1px solid var(--color-principal);
36   border-radius: 6px;
37   background: white;
38   color: var(--color-principal);
39   cursor: pointer;
40 }
41
42 button:hover {
43   background: var(--color-principal-suave);
44 }
45
46 button:disabled {
47   opacity: 0.45;
48   cursor: not-allowed;
49 }
50
51 input,
52 select,
53 textarea {
54   font: inherit;
55   padding: 6px 8px;
56   border: 1px solid #c5cad3;
57   border-radius: 6px;
58 }
59
60 pre {
61   overflow-x: auto;
62 }

```

Las clases que varios componentes comparten viven aparte:

src/styles/utilidades.css

```

1  /* Clases de uso general, compartidas por varios componentes. */
2
3  .caja {
4    background: white;
5    border: 1px solid var(--color-borde);
6    border-radius: var(--radio);
7    padding: var(--espacio) 16px;
8    margin: 10px 0;
9  }
10
11  .fila {

```

```

12   display: flex;
13   gap: var(--espacio);
14   flex-wrap: wrap;
15   align-items: center;
16 }
17
18 .formulario {
19   display: flex;
20   flex-direction: column;
21   gap: 6px;
22   max-width: 380px;
23 }
24
25 .alerta {
26   color: var(--color-alerta);
27   font-weight: 600;
28 }
29
30 .ok {
31   color: var(--color-ok);
32   font-weight: 600;
33 }
34
35 .badge {
36   background: var(--color-peligro);
37   color: white;
38   border-radius: 12px;
39   padding: 2px 10px;
40   font-size: 0.85rem;
41 }

```

Ambos archivos se cargan una sola vez, en el punto de entrada:

```

1 import './index.css'
2 import './styles/utilidades.css'

```

10.4 CSS por componente

Cada componente con estilos propios tiene un archivo con su mismo nombre, en la misma carpeta, y lo importa en la primera línea.

src/components/props/Boton.css

```

1  /* Clase base más una clase por variante: boton-primario, boton-peligro... */
2  .boton {
3    transition: background 0.15s;
4  }
5
6  .boton-primario {
7    background: var(--color-principal);
8    color: white;
9  }
10
11 .boton-primario:hover {
12   background: #066880;
13 }
14
15 .boton-peligro {

```

```

16   border-color: var(--color-peligro);
17   color: var(--color-peligro);
18 }
19
20 .boton-peligro:hover {
21   background: #fce7f0;
22 }

```

src/components/props/Boton.jsx

```

1 import './Boton.css'
2
3 function Boton({ texto, variante = 'normal', deshabilitado = false, onClick }) {
4   return (
5     <button className={`boton boton-${variante}`} ...>{texto}</button>
6   )
7 }

```

El patrón de una clase base más una clase por variante (boton boton-primario) es habitual: la base define lo común y la variante solo lo que cambia.

10.5 CSS Modules

Un archivo que termina en `.module.css` se importa como un objeto. Cada propiedad de ese objeto es el nombre real que tendrá la clase en el navegador, generado por Vite.

src/components/listas/Buscador.module.css

```

1  /* CSS Module: Vite renombra estas clases al compilar (buscador -> _buscador_1a2b3)
2     para que no choquen con clases de otros archivos. */
3  .controles {
4     display: flex;
5     gap: var(--espacio);
6     flex-wrap: wrap;
7  }
8
9  .resumen {
10     color: var(--color-tenue);
11     font-size: 0.9rem;
12  }
13
14  .lista {
15     list-style: none;
16     padding: 0;
17     margin: 0;
18  }
19
20  .lista li {
21     padding: 6px 0;
22     border-bottom: 1px solid var(--color-borde);
23  }
24
25  .vacio {
26     color: var(--color-alerta);
27     font-weight: 600;
28  }

```

```

1 import estilos from './Buscador.module.css'

```

```

2 // ...
3 <div className={estilos.controles}>
4 <p className={estilos.resumen}>...</p>

```

En pantalla, `controles` se convierte en algo como `_controles_18kl5_3`. Como ese nombre es único, dos componentes pueden tener una clase lista sin interferirse. El costo es escribir `estilos.` en cada clase.

Nota

Los nombres con guion, como `.mi-clase`, se leen con corchetes: `estilos['mi-clase']`. Para evitarlo, en los CSS Modules se suele escribir en una sola palabra o en camelCase.

10.6 Clases dinámicas

`className` recibe un texto, así que se arma con las mismas herramientas de JavaScript de siempre:

```

1 // Ternario: una clase u otra
2 <p className={aprobado ? 'ok' : 'alerta'}>{mensaje}</p>
3
4 // Clase fija mas una condicional
5 <div className={encima ? 'zona zona-activa' : 'zona'}>...</div>
6
7 // Template literal con un valor variable
8 <button className={`boton boton-${variante}`}>...</button>
9
10 // Varias condiciones: arreglo, filtrado y unido por espacios
11 <li className={['item', hecha && 'item-hecha', urgente && 'item-urgente']
12   .filter(Boolean).join(' ')}>

```

El `.filter(Boolean)` descarta los `false` antes de unir, para que no queden espacios sobrantes.

10.7 Estilos en línea

El atributo `style` recibe un **objeto**, no un texto. Las propiedades van en camelCase y los valores como texto, salvo los números en píxeles:

```

1 <p style={{ color: 'white', backgroundColor: '#087EA4', padding: 8 }}>Destacado</p>
2
3 // Util cuando el valor se calcula
4 <div className="barra" style={{ width: `${porcentaje}%` }} />

```

Las dobles llaves confunden al principio: las externas abren JavaScript dentro de JSX y las internas son las del objeto. Úsalos solo para valores que cambian en tiempo de ejecución; el resto va en CSS, donde puedes usar `:hover`, `@media` y animaciones, cosa que los estilos en línea no permiten.

10.8 Cómo quedan los archivos

```

src/
|-- index.css                variables y reglas base
|-- styles/utilidades.css    clases compartidas (.caja, .fila, .alerta)
|-- App.css                  amazon: cabecera y pestanas
`-- components/
    |-- Ejemplo.css
    |-- basicos/Perfil.css, Tarjeta.css

```

```
|-- jsx/Composicion.css, TarjetaPresentacion.css
|-- props/Boton.css, ListaProductos.css
|-- eventos/EventosComunes.css
|-- listas/Buscador.module.css      (CSS Module)
|-- formularios/Inscripcion.css
|-- efectos/Reloj.css
|-- render/DemoRender.css
|-- tareas/ListaTareas.css
```

En `npm run dev`, Vite inyecta los estilos sin recargar la página: guardas el `.css` y el cambio se ve al instante. En `npm run build`, junta todo en un solo archivo `.css` dentro de `dist/assets/`.

10.9 Errores frecuentes con estilos

Síntoma	Causa y solución
El estilo no se aplica	Escribiste <code>class</code> en vez de <code>className</code> .
<i>Objects are not valid as a React child</i>	Pusiste el objeto de estilos donde va el contenido. Revisa que sea <code>style={{...}}</code> .
El estilo en línea no funciona	Propiedad con guion: se escribe <code>backgroundColor</code> , no <code>background-color</code> .
Un componente le cambia el aspecto a otro	Dos archivos usan el mismo nombre de clase. Prefija las clases o usa un CSS Module.
<code>estilos.miClase</code> es undefined	El archivo no termina en <code>.module.css</code> , o el nombre de la clase no coincide.
El CSS no aparece en producción	Ningún archivo lo importa. Vite solo incluye lo que se importa desde el código.

Ejercicio de estilos 1

Cambia `--color-principal` en `index.css` por otro color y recarga. Fíjate cuántas partes de la aplicación cambian con esa sola línea.

Ejercicio de estilos 2

Crea `ItemTarea.module.css` y aplícalo al componente `ItemTarea`: una clase para la fila y otra para el texto tachado, en vez del estilo en línea que usa hoy.

11 Props: pasar datos a un componente

Las **props** son los parámetros de un componente. Se entregan como atributos y llegan a la función dentro de un objeto, que normalmente se desestructura.

```
src/components/Tarjeta.jsx
function Tarjeta({ titulo, precio, disponible = true }) {
  return (
    <article className="tarjeta">
      <h3>{titulo}</h3>
      <p>${precio.toLocaleString('es-CL')}</p>
      {!disponible && <small>Sin stock</small>}
    </article>
```

```

8   )
9   }
10
11  export default Tarjeta

```

```

1  <Tarjeta titulo="Teclado" precio={25990} />
2  <Tarjeta titulo="Mouse" precio={12990} disponible={false} />

```

Observa que los textos van entre comillas y los números, booleanos, arreglos u objetos van entre llaves.

Atención

Las props son de solo lectura. Un componente nunca debe modificar las props que recibe. Si un dato necesita cambiar, se usa estado.

11.1 La prop especial children

Lo que escribes entre la etiqueta de apertura y la de cierre llega en la prop children. Sirve para componentes contenedores:

```

1  function Caja({ children }) {
2    return <div className="caja">{children}</div>
3  }
4
5  <Caja>
6    <p>Cualquier contenido</p>
7  </Caja>

```

11.2 Tipos de valores en props

```

1  <Ficha
2    nombre="Ana"                // texto
3    edad={21}                  // numero
4    activa={true}              // booleano
5    cursos={['React', 'SQL']}  // arreglo
6    direccion={{ ciudad: 'Quilpue' }} // objeto (llaves externas de JSX, internas del
    objeto)
7    alGuardar={guardar}        // funcion
8  />

```

Una prop escrita sin valor, como `<Boton deshabilitado />`, equivale a `deshabilitado={true}`.

11.3 Más ejemplos

Un componente reutilizable. Todo lo que cambia entre un botón y otro llega por props. La función `onClick` también es una prop: el padre decide qué pasa al hacer clic.

src/components/props/Boton.jsx

```

1  // Componente reutilizable: todo lo que cambia entre un botón y otro llega por props.
2  function Boton({ texto, variante = 'normal', deshabilitado = false, onClick }) {
3    return (
4      <button className={`boton boton-${variante}`} disabled={deshabilitado} onClick={
        onClick}>

```

```

5     {texto}
6   </button>
7 )
8 }
9
10 export default Boton

```

```

src/components/props/PanelBotones.jsx
1 import Boton from './Boton'
2
3 function PanelBotones() {
4   const guardar = () => alert('Guardado')
5   const borrar = () => alert('Borrado')
6
7   return (
8     <div className="caja">
9       <Boton texto="Guardar" variante="primario" onClick={guardar} />
10      <Boton texto="Borrar" variante="peligro" onClick={borrar} />
11      /* Escribir solo "deshabilitado" equivale a deshabilitado={true} */
12      <Boton texto="No disponible" deshabilitado />
13    </div>
14  )
15 }
16
17 export default PanelBotones

```

Un objeto completo como prop. Cuando un dato tiene muchas propiedades, es más cómodo entregar el objeto entero y desestructurarlo dentro del hijo.

```

src/components/props/ListaProductos.jsx
1 const productos = [
2   { id: 1, nombre: 'Notebook', precio: 549990, categoria: 'Computación' },
3   { id: 2, nombre: 'Audífonos', precio: 34990, categoria: 'Audio' },
4   { id: 3, nombre: 'Webcam', precio: 42990, categoria: 'Computación' },
5 ]
6
7 // Recibe un objeto completo como prop y lo desestructura dentro.
8 function TarjetaProducto({ producto }) {
9   const { nombre, precio, categoria } = producto
10  return (
11    <article className="tarjeta">
12      <small>{categoria}</small>
13      <h3>{nombre}</h3>
14      <p>${precio.toLocaleString('es-CL')}</p>
15    </article>
16  )
17 }
18
19 function ListaProductos() {
20  return (
21    <div className="fila">
22      {productos.map((p) => (
23        <TarjetaProducto key={p.id} producto={p} />
24      ))}
25    </div>
26  )
27 }
28

```

28 **export default** ListaProductos**Nota**

Otra forma de entregar todas las propiedades de un objeto como props separadas es el spread: `<Tarjeta {...p} />` equivale a `<Tarjeta id={p.id} nombre={p.nombre} ... />`. Es práctico, pero hace menos evidente qué recibe el componente.

12 Estado con `useState`

El **estado** es la memoria de un componente. Cuando el estado cambia, React vuelve a ejecutar la función del componente y actualiza la pantalla.

src/components/Contador.jsx

```
1 import { useState } from 'react'
2
3 function Contador() {
4   const [cuenta, setCuenta] = useState(0)
5
6   return (
7     <div>
8       <p>Has hecho clic {cuenta} veces</p>
9       <button onClick={() => setCuenta(cuenta + 1)}>Sumar</button>
10      <button onClick={() => setCuenta(0)}>Reiniciar</button>
11    </div>
12  )
13 }
14
15 export default Contador
```

`useState(0)` devuelve un arreglo de dos elementos: el valor actual y la función que lo cambia. Por convención se nombran algo y `setAlgo`.

12.1 Tres reglas que evitan errores

1. **Nunca modifiques el estado directamente.** `cuenta = cuenta + 1` no actualiza la pantalla. Usa siempre la función `set`.
2. **Con arreglos y objetos, crea uno nuevo.** En vez de `lista.push(x)`, usa `setLista([...lista, x])`. En vez de `obj.nombre = 'x'`, usa `setObj({ ...obj, nombre: 'x' })`.
3. **Si el nuevo valor depende del anterior**, usa la forma con función: `setCuenta(c => c + 1)`. Garantiza que trabajas con el valor más reciente.

Atención

Los *hooks* (funciones que empiezan con `use`) solo se llaman en el nivel superior del componente. Nunca dentro de un `if`, un ciclo o una función interna.

12.2 ¿Props o estado?

Props	Estado
Vienen del componente padre.	Pertenece al propio componente.
El componente no las puede cambiar.	El componente lo cambia con su función <code>set</code> .
Sirven para configurar un componente.	Sirve para recordar lo que cambia con el uso.

Pregúntate: ¿este dato cambia mientras el usuario usa la página? Si no cambia, no es estado. ¿Se puede calcular a partir de otro estado o de las props? Entonces tampoco es estado: es un **valor derivado** y se calcula en el cuerpo del componente.

12.3 Más ejemplos

Estado booleano. El caso más común: mostrar u ocultar algo. `!visible` invierte el valor.

src/components/estado/Interruptor.jsx

```

1 import { useState } from 'react'
2
3 function Interruptor() {
4   const [visible, setVisible] = useState(false)
5
6   return (
7     <div className="caja">
8       <button onClick={() => setVisible(!visible)}>
9         {visible ? 'Ocultar' : 'Mostrar'} detalle
10      </button>
11      {visible && <p>Este texto aparece y desaparece según el estado.</p>}
12    </div>
13  )
14 }
15
16 export default Interruptor

```

Estado de texto y valor derivado. restantes no usa `useState`: se calcula en cada render a partir de texto. Guardarlo en un segundo estado obligaría a mantener dos valores sincronizados y es fuente de errores.

src/components/estado/ContadorCaracteres.jsx

```

1 import { useState } from 'react'
2
3 const MAXIMO = 80
4
5 function ContadorCaracteres() {
6   const [texto, setTexto] = useState('')
7
8   // Valor derivado: se calcula a partir del estado, no necesita su propio useState
9   const restantes = MAXIMO - texto.length
10
11   return (
12     <div className="caja formulario">
13       <textarea
14         rows={3}
15         maxLength={MAXIMO}

```

```

16     placeholder="Escribe un mensaje"
17     value={texto}
18     onChange={(e) => setTexto(e.target.value)}
19   />
20   <p className={restantes < 10 ? 'alerta' : ''}>Quedan {restantes} caracteres</p>
21   <p>Vista previa en mayúsculas: {texto.toUpperCase()}</p>
22 </div>
23 )
24 }
25
26 export default ContadorCaracteres

```

Estado con un objeto. Para cambiar una propiedad se crea un objeto nuevo copiando el anterior con `...perfil` y reemplazando solo lo que cambia. Si escribieras `perfil.edad++`, la pantalla no se actualizaría.

src/components/estado/PerfilEditable.jsx

```

1 import { useState } from 'react'
2
3 function PerfilEditable() {
4   const [perfil, setPerfil] = useState({ nombre: 'Ana', ciudad: 'Viña del Mar', edad: 20
5   })
6
7   // Se copia el objeto con ...perfil y se reemplaza solo la propiedad que cambia
8   const cumplirAños = () => setPerfil({ ...perfil, edad: perfil.edad + 1 })
9   const mudarse = () => setPerfil({ ...perfil, ciudad: 'Valparaíso' })
10
11   return (
12     <div className="caja">
13       <p>
14         {perfil.nombre}, {perfil.edad} años, vive en {perfil.ciudad}
15       </p>
16       <button onClick={cumplirAños}>Cumplir años</button>
17       <button onClick={mudarse}>Mudarse a Valparaíso</button>
18     </div>
19   )
20 }
21
22 export default PerfilEditable

```

Estado con un arreglo. Agregar con `spread`, quitar con `slice` o `filter`, vaciar con un arreglo vacío. La suma es otro valor derivado.

src/components/estado/NumerosAleatorios.jsx

```

1 import { useState } from 'react'
2
3 function NumerosAleatorios() {
4   const [numeros, setNumeros] = useState([])
5
6   const agregar = () => {
7     const nuevo = Math.floor(Math.random() * 100) + 1
8     setNumeros([...numeros, nuevo]) // arreglo nuevo con el elemento al final
9   }
10
11   const quitarUltimo = () => setNumeros(numeros.slice(0, -1))
12   const vaciar = () => setNumeros([])
13
14   const suma = numeros.reduce((acumulado, n) => acumulado + n, 0)

```

```

15
16   return (
17     <div className="caja">
18       <button onClick={agregar}>Agregar número</button>
19       <button onClick={quitarUltimo} disabled={numeros.length === 0}>
20         Quitar último
21       </button>
22       <button onClick={vaciar}>Vaciar</button>
23       <p>Números: {numeros.length > 0 ? numeros.join(', ') : 'ninguno'}</p>
24       <p>Suma: {suma}</p>
25     </div>
26   )
27 }
28
29 export default NumerosAleatorios

```

Operación	Forma correcta	Evitar
Agregar al final	setLista([...lista, x])	lista.push(x)
Agregar al inicio	setLista([x, ...lista])	lista.unshift(x)
Eliminar	setLista(lista.filter(e => e.id !== id))	lista.splice(i, 1)
Modificar uno	setLista(lista.map(e => e.id === id ? {...e, hecho: true} : e))	lista[i].hecho = true
Ordenar	setLista([...lista].sort(f))	lista.sort(f)

13 Eventos

Los eventos se asignan con atributos como `onClick`, `onChange` o `onSubmit`, y reciben una **función**, no el resultado de llamarla.

```

1 function Botones() {
2   const saludar = () => alert('Hola')
3   const saludarA = (nombre) => alert(`Hola, ${nombre}`)
4
5   return (
6     <>
7       <button onClick={saludar}>Correcto</button>
8       <button onClick={() => saludarA('Ana')}>Correcto con argumento</button>
9       <button onClick={saludar()}>Incorrecto: se ejecuta al renderizar</button>
10    </>
11  )
12 }

```

13.1 El objeto evento

La función que maneja un evento recibe un parámetro, normalmente llamado `e`, con información de lo ocurrido:

- `e.target`: el elemento que generó el evento. `e.target.value` es el texto de un input.
- `e.key`: la tecla presionada, en eventos de teclado ('Enter', 'Escape', 'a').

- `e.preventDefault()`: cancela el comportamiento por defecto del navegador, como recargar la página al enviar un formulario.

13.2 Eventos más usados

Evento	Se dispara cuando
<code>onClick</code>	Se hace clic en el elemento.
<code>onDoubleClick</code>	Se hace doble clic.
<code>onChange</code>	Cambia el valor de un input, select o textarea. En React ocurre con cada tecla.
<code>onSubmit</code>	Se envía un formulario (botón submit o Enter en un campo).
<code>onKeyDown</code>	Se presiona una tecla con el foco en el elemento.
<code>onFocus</code> / <code>onBlur</code>	El elemento recibe o pierde el foco.
<code>onMouseEnter</code> / <code>onMouseLeave</code>	El puntero entra o sale del elemento.

13.3 Más ejemplos

src/components/eventos/EventosComunes.jsx

```

1 import { useState } from 'react'
2
3 function EventosComunes() {
4   const [ultimo, setUltimo] = useState('ninguno todavía')
5   const [encima, setEncima] = useState(false)
6
7   // El objeto evento (e) trae información: qué tecla, qué elemento, qué valor
8   const alPresionarTecla = (e) => {
9     if (e.key === 'Enter') {
10       setUltimo(`Enter con el texto "${e.target.value}"`)
11     }
12   }
13
14   return (
15     <div className="caja formulario">
16       <p>
17         Último evento: <strong>{ultimo}</strong>
18       </p>
19
20       <div
21         className={encima ? 'zona zona-activa' : 'zona'}
22         onMouseEnter={() => setEncima(true)}
23         onMouseLeave={() => setEncima(false)}
24         onDoubleClick={() => setUltimo('doble clic en la zona')}
25       >
26         {encima ? 'El mouse está encima' : 'Pasa el mouse o haz doble clic aquí'}
27       </div>
28
29       <input placeholder="Escribe y presiona Enter" onKeyDown={alPresionarTecla} />
30
31       <input
32         placeholder="Haz clic aquí y luego fuera"
33         onFocus={() => setUltimo('foco en el campo')}
34         onBlur={() => setUltimo('el campo perdió el foco')}
35       />
36
37       <input

```

```

38     type="range"
39     min="0"
40     max="100"
41     defaultValue="50"
42     onChange={e => setUltimo(`rango en ${e.target.value}`)}
43   />
44 </div>
45 )
46 }
47
48 export default EventosComunes

```

El input `type="range"` usa `defaultValue` en lugar de `value`: es un input **no controlado**, React no guarda su valor y solo escucha sus cambios. Para la mayoría de los formularios conviene la versión controlada de la sección 16.

Convención de nombres. Las props que reciben funciones suelen empezar con `on` (`onAgregar`, `onEliminar`) y las funciones que las manejan con un verbo o con `manejar` (`manejarEnvio`, `alPresionarTecla`).

14 Renderizado condicional

Como JSX es JavaScript, las condiciones se escriben con operadores del lenguaje.

```

1 function Estado({ conectado, mensajes }) {
2   // 1. if con return anticipado
3   if (!conectado) return <p>Inicia sesion para continuar</p>
4
5   return (
6     <div>
7       /* 2. Operador ternario: una cosa u otra */
8       <p>{mensajes > 0 ? 'Tienes mensajes nuevos' : 'Bandeja vacia'}</p>
9
10      /* 3. Operador &&: mostrar o no mostrar */
11      {mensajes > 0 && <span className="badge">{mensajes}</span>}
12    </div>
13  )
14 }

```

Atención

Con `&&`, si el valor de la izquierda es el número 0, React muestra un “0” en pantalla. Por eso conviene escribir `mensajes > 0 &&` y no solo `mensajes &&`.

14.1 Más ejemplos

Mostrar u ocultar una contraseña. La condición cambia un atributo (`type`) y el texto del botón. El nivel de seguridad se calcula con `if` antes del `return`, porque dentro del JSX no se puede.

src/components/condicional/Contrasena.jsx

```

1 import { useState } from 'react'
2
3 function Contraseña() {

```

```

4   const [clave, setClave] = useState('')
5   const [mostrar, setMostrar] = useState(false)
6
7   // Condiciones con if antes del return
8   let seguridad = 'Débil'
9   if (clave.length >= 8 && /\d/.test(clave)) {
10    seguridad = 'Fuerte'
11  } else if (clave.length >= 6) {
12    seguridad = 'Media'
13  }
14
15  return (
16    <div className="caja">
17      <input
18        type={mostrar ? 'text' : 'password'}
19        placeholder="Contraseña"
20        value={clave}
21        onChange={e => setClave(e.target.value)}
22      />
23      <button onClick={() => setMostrar(!mostrar)}>{mostrar ? 'Ocultar' : 'Mostrar'}</button>
24      {clave.length > 0 && <p>Seguridad: {seguridad}</p>}
25    </div>
26  )
27 }
28
29 export default Contraseña

```

Varias condiciones. Cuando hay más de dos casos, los ternarios anidados se vuelven ilegibles. Es mejor mover la lógica a una función auxiliar.

src/components/condicional/Calificacion.jsx

```

1  import { useState } from 'react'
2
3  // Función auxiliar fuera del componente: recibe una nota y devuelve un texto
4  function mensajePara(nota) {
5    if (nota >= 6) return 'Excelente'
6    if (nota >= 4) return 'Aprobado'
7    return 'Reprobado'
8  }
9
10 function Calificacion() {
11   const [nota, setNota] = useState(5.5)
12   const aprobado = nota >= 4
13
14   return (
15     <div className="caja formulario">
16       <label htmlFor="nota">Nota: {nota.toFixed(1)}</label>
17       /* Los inputs entregan texto: Number() lo convierte a número */
18       <input
19         id="nota"
20         type="range"
21         min="1"
22         max="7"
23         step="0.1"
24         value={nota}
25         onChange={e => setNota(Number(e.target.value))}
26       />
27       <p className={aprobado ? 'ok' : 'alerta'}>{mensajePara(nota)}</p>

```

```

28     </div>
29   )
30 }
31
32 export default Calificacion

```

Fíjate también en `className={aprobado ? 'ok' : 'alerta'}`: una condición puede decidir qué clase CSS aplicar, y así cambiar el color sin tocar el contenido.

Situación	Qué usar
Mostrar algo o nada	<code>{condicion && <Elemento />}</code>
Una cosa u otra	<code>{condicion ? <A /> : }</code>
Tres o más casos	Función auxiliar o <code>if</code> antes del <code>return</code>
No mostrar el componente entero	<code>if (...) return null</code> al inicio

15 Listas y la prop key

Para mostrar un arreglo se usa `map`, que transforma cada dato en un elemento JSX. Cada elemento necesita una prop `key` única y estable, que React usa para saber qué cambió.

```

1  const productos = [
2    { id: 1, nombre: 'Teclado', precio: 25990 },
3    { id: 2, nombre: 'Mouse', precio: 12990 },
4    { id: 3, nombre: 'Monitor', precio: 149990 },
5  ]
6
7  function Catalogo() {
8    return (
9      <ul>
10        {productos.map((p) => (
11          <li key={p.id}>
12            {p.nombre}: ${p.precio}
13          </li>
14        ))}
15      </ul>
16    )
17  }

```

Nota

Evita usar el índice del arreglo como `key` cuando la lista puede reordenarse o perder elementos. Usa un identificador propio del dato, como `id`. Para filtrar, combina `filter` y luego `map`.

15.1 Por qué React necesita key

Si una lista pasa de `[Ana, Luis]` a `[Pedro, Ana, Luis]`, React tiene que saber si Ana cambió de nombre a Pedro o si Pedro es nuevo. Con `key` lo sabe: compara las claves, ve que las de Ana y Luis siguen ahí y crea solo un elemento nuevo. Sin claves estables, un input dentro de la lista podría quedar asociado a la fila equivocada.

La `key` va en el elemento que devuelve el `map`, no en el componente hijo por dentro, y no llega como prop: si el hijo necesita el `id`, pásalo aparte.

15.2 Más ejemplos

Buscar, filtrar y ordenar. Los datos originales no cambian. En cada render se calcula la lista que se muestra a partir del texto buscado y del orden elegido, que sí son estado.

src/components/listas/Buscador.jsx

```

1  import { useState } from 'react'
2
3  const cursos = [
4    { id: 1, nombre: 'Fundamentos de Programación', credits: 10 },
5    { id: 2, nombre: 'Programación de Base de Datos', credits: 8 },
6    { id: 3, nombre: 'Desarrollo Fullstack', credits: 12 },
7    { id: 4, nombre: 'Arquitectura de Software', credits: 6 },
8    { id: 5, nombre: 'Programación Web', credits: 8 },
9    { id: 6, nombre: 'Estadística', credits: 6 },
10 ]
11
12 function Buscador() {
13   const [busqueda, setBusqueda] = useState('')
14   const [orden, setOrden] = useState('nombre')
15
16   // 1. Filtrar según lo escrito, sin distinguir mayúsculas
17   const texto = busqueda.toLowerCase()
18   const filtrados = cursos.filter((c) => c.nombre.toLowerCase().includes(texto))
19
20   // 2. Ordenar una copia: sort modifica el arreglo sobre el que se llama
21   const ordenados = [...filtrados].sort((a, b) =>
22     orden === 'nombre' ? a.nombre.localeCompare(b.nombre) : b.credits - a.credits,
23   )
24
25   return (
26     <div className="caja">
27       <div className="fila">
28         <input placeholder="Buscar curso" value={busqueda} onChange={(e) => setBusqueda(e
29           .target.value)} />
30         <select value={orden} onChange={(e) => setOrden(e.target.value)}>
31           <option value="nombre">Ordenar por nombre</option>
32           <option value="credits">Ordenar por créditos</option>
33         </select>
34       </div>
35
36       <p>
37         {ordenados.length} de {cursos.length} cursos
38       </p>
39
40       {ordenados.length === 0 ? (
41         <p className="alerta">No hay coincidencias</p>
42       ) : (
43         <ul>
44           {ordenados.map((c) => (
45             <li key={c.id}>
46               {c.nombre} ({c.credits} créditos)
47             </li>
48           ))}
49         </ul>
50       )}
51     </div>
52   )
53 }
```

export default Buscador

El patrón es siempre el mismo: **datos originales** → **filter** → **sort** sobre una copia → **map** a JSX. Además se muestra un mensaje cuando la lista queda vacía, algo que el usuario agradece.

16 Formularios controlados

Un input **controlado** guarda su valor en el estado. El valor mostrado viene del estado y cada tecla lo actualiza.

src/components/Registro.jsx

```

1 import { useState } from 'react'
2
3 function Registro() {
4   const [form, setForm] = useState({ nombre: '', correo: '' })
5
6   const manejarCambio = (e) => {
7     const { name, value } = e.target
8     setForm({ ...form, [name]: value })
9   }
10
11   const manejarEnvio = (e) => {
12     e.preventDefault() // evita que la pagina se recargue
13     if (!form.nombre || !form.correo) {
14       alert('Completa todos los campos')
15       return
16     }
17     console.log('Datos enviados:', form)
18     setForm({ nombre: '', correo: '' })
19   }
20
21   return (
22     <form onSubmit={manejarEnvio}>
23       <label htmlFor="nombre">Nombre</label>
24       <input id="nombre" name="nombre" value={form.nombre} onChange={manejarCambio} />
25
26       <label htmlFor="correo">Correo</label>
27       <input id="correo" name="correo" type="email"
28         value={form.correo} onChange={manejarCambio} />
29
30       <button type="submit">Registrar</button>
31     </form>
32   )
33 }
34
35 export default Registro

```

El truco de `[name]: value` permite usar una sola función para todos los campos, siempre que cada input tenga un name igual a la propiedad del objeto.

16.1 Cada tipo de campo

Campo	Se controla con	Detalle
Texto, email, número	value + onChange	El valor siempre es texto, incluso con type="number".
Textarea	value + onChange	En HTML el texto va entre etiquetas; en React, en value.
Select	value + onChange	El value va en el select, no en cada option.
Radio	checked + onChange	Todos comparten name; checked compara con el estado.
Checkbox	checked + onChange	Se lee e.target.checked, que es booleano.

16.2 Más ejemplos

Todos los campos y validación. Una sola función cambiar maneja todos los campos. La validación devuelve un objeto de errores; si está vacío, el formulario se envía. Los mensajes se muestran con && bajo cada campo.

src/components/formularios/Inscripcion.jsx

```

1 import { useState } from 'react'
2
3 const inicial = { nombre: '', jornada: 'diurna', sede: '', comentario: '', acepta: false
4   }
5
6 // Devuelve un objeto con un mensaje por cada campo que tenga problemas
7 function validar(datos) {
8   const errores = {}
9   if (datos.nombre.trim().length < 3) errores.nombre = 'Escribe al menos 3 letras'
10  if (datos.sede === '') errores.sede = 'Elige una sede'
11  if (!datos.acepta) errores.acepta = 'Debes aceptar las condiciones'
12  return errores
13 }
14
15 function Inscripcion() {
16   const [datos, setDatos] = useState(inicial)
17   const [errores, setErrores] = useState({})
18   const [enviado, setEnviado] = useState(null)
19
20   // Una sola función para todos los campos: checkbox usa checked, el resto value
21   const cambiar = (e) => {
22     const { name, value, type, checked } = e.target
23     setDatos({ ...datos, [name]: type === 'checkbox' ? checked : value })
24   }
25
26   const enviar = (e) => {
27     e.preventDefault()
28     const encontrados = validar(datos)
29     setErrores(encontrados)
30     if (Object.keys(encontrados).length > 0) return
31     setEnviado(datos)
32     setDatos(inicial)
33   }
34
35   return (

```

```

35 </>
36 <form className="caja formulario" onSubmit={enviar} noValidate>
37   <label htmlFor="ins-nombre">Nombre</label>
38   <input id="ins-nombre" name="nombre" value={datos.nombre} onChange={cambiar} />
39   {errores.nombre && <small className="alerta">{errores.nombre}</small>}
40
41   <span>Jornada</span>
42   <label>
43     <input type="radio" name="jornada" value="diurna"
44       checked={datos.jornada === 'diurna'} onChange={cambiar} />
45     Diurna
46   </label>
47   <label>
48     <input type="radio" name="jornada" value="vespertina"
49       checked={datos.jornada === 'vespertina'} onChange={cambiar} />
50     Vespertina
51   </label>
52
53   <label htmlFor="ins-sede">Sede</label>
54   <select id="ins-sede" name="sede" value={datos.sede} onChange={cambiar}>
55     <option value="">Selecciona...</option>
56     <option value="vina">Viña del Mar</option>
57     <option value="valparaiso">Valparaíso</option>
58     <option value="santiago">Santiago</option>
59   </select>
60   {errores.sede && <small className="alerta">{errores.sede}</small>}
61
62   <label htmlFor="ins-comentario">Comentario (opcional)</label>
63   <textarea id="ins-comentario" name="comentario" rows={3}
64     value={datos.comentario} onChange={cambiar} />
65
66   <label>
67     <input type="checkbox" name="acepta" checked={datos.acepta} onChange={cambiar}
68   />
69     Acepto las condiciones
70   </label>
71   {errores.acepta && <small className="alerta">{errores.acepta}</small>}
72
73   <button type="submit">Inscribirme</button>
74 </form>
75
76   {enviado && <pre className="caja">{JSON.stringify(enviado, null, 2)}</pre>}
77 </>
78 )
79 }
80 export default Incripcion

```

`noValidate` desactiva la validación propia del navegador para que se vean los mensajes del componente. En un proyecto real la validación también debe hacerse en el servidor.

17 Efectos con `useEffect`

`useEffect` ejecuta código **después** de que el componente se muestra. Se usa para sincronizarse con algo externo: pedir datos a una API, usar temporizadores o escuchar eventos del navegador.

```

1  useEffect(() => {
2    // código del efecto
3    return () => {
4      // limpieza opcional (se ejecuta al desmontar o antes de repetir)
5    }
6  }, [dependencias])

```

El arreglo de dependencias decide cuándo se ejecuta:

Dependencias	Cuándo se ejecuta
[]	Una sola vez, al montar el componente.
[a, b]	Al montar y cada vez que cambie a o b.
(sin arreglo)	Después de cada renderizado. Rara vez es lo que quieres.

17.1 Ejemplo: consumir una API

```

src/components/Usuarios.jsx

import { useState, useEffect } from 'react'

function Usuarios() {
  const [usuarios, setUsuarios] = useState([])
  const [cargando, setCargando] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then((res) => {
        if (!res.ok) throw new Error('Error ' + res.status)
        return res.json()
      })
      .then((datos) => setUsuarios(datos))
      .catch((err) => setError(err.message))
      .finally(() => setCargando(false))
  }, [])

  if (cargando) return <p>Cargando...</p>
  if (error) return <p>Ocurrió un error: {error}</p>

  return (
    <ul>
      {usuarios.map((u) => (
        <li key={u.id}>{u.name} {u.email}</li>
      ))}
    </ul>
  )
}

export default Usuarios

```

Atención

Si dentro del efecto actualizas un estado que también está en las dependencias, generas un ciclo infinito de renderizados. Revisa siempre qué pones en el arreglo.

17.2 Cuándo usar `useEffect` y cuándo no

`useEffect` sirve para sincronizar el componente con algo que está **fuera de React**: el título de la pestaña, un temporizador, una API, `localStorage`. No lo necesitas para:

- Calcular un valor a partir del estado. Hazlo directamente en el cuerpo del componente.
- Responder a un clic. Eso va en el manejador del evento.

17.3 Más ejemplos

Efecto con dependencia. El título de la pestaña se actualiza cada vez que cambia `clics`. La limpieza restaura el título al cambiar de sección.

src/components/efectos/TituloDocumento.jsx

```
1 import { useState, useEffect } from 'react'
2
3 function TituloDocumento() {
4   const [clics, setClics] = useState(0)
5
6   // Se ejecuta al montar y cada vez que cambia "clics"
7   useEffect(() => {
8     document.title = `Clics: ${clics}`
9     // Limpieza: al salir de esta pestaña se restaura el título original
10    return () => {
11      document.title = 'Guía React: primeros pasos'
12    }
13  }, [clics])
14
15  return (
16    <div className="caja">
17      <p>Mira el título de la pestaña del navegador.</p>
18      <button onClick={() => setClics(clics + 1)}>Clic ({clics})</button>
19    </div>
20  )
21 }
22
23 export default TituloDocumento
```

Temporizador y limpieza. Un `setInterval` sigue corriendo aunque el componente desaparezca, a menos que lo detengas. La función que devuelve el efecto se ejecuta al desmontar. Abre la consola y presiona “Desmontar” para verlo.

src/components/efectos/Reloj.jsx

```
1 import { useState, useEffect } from 'react'
2
3 function Reloj() {
4   const [hora, setHora] = useState(new Date())
5
6   useEffect(() => {
7     const id = setInterval(() => setHora(new Date()), 1000)
8     console.log('Reloj montado: intervalo iniciado')
9     // Sin esta limpieza el intervalo seguiría corriendo después de desmontar
10    return () => {
11      clearInterval(id)
12      console.log('Reloj desmontado: intervalo detenido')
13    }
14  })
15 }
```

```

13     }
14   }, [])
15
16   return <p className="reloj">{hora.toLocaleTimeString('es-CL')}</p>
17 }
18
19 function DemoReloj() {
20   const [activo, setActivo] = useState(true)
21
22   return (
23     <div className="caja">
24       <button onClick={() => setActivo(!activo)}>{activo ? 'Desmontar' : 'Montar'} reloj
25     </button>
26     {activo && <Reloj />}
27   </div>
28 )
29 }
30
31 export default DemoReloj

```

Pedir datos según una selección, con `async/await`. El efecto no puede ser `async` directamente, así que se define una función `async` dentro y se llama. Como `usuarioId` está en las dependencias, la petición se repite cada vez que eliges otro usuario. La variable `cancelado` evita que una respuesta lenta de una selección anterior sobrescriba la actual.

src/components/efectos/PostsPorUsuario.jsx

```

1 import { useState, useEffect } from 'react'
2
3 function PostsPorUsuario() {
4   const [usuarioId, setUsuarioId] = useState(1)
5   const [posts, setPosts] = useState([])
6   const [cargando, setCargando] = useState(true)
7   const [error, setError] = useState(null)
8
9   useEffect(() => {
10     let cancelado = false // evita guardar la respuesta de una petición vieja
11
12     const cargar = async () => {
13       setCargando(true)
14       setError(null)
15       try {
16         const url = `https://jsonplaceholder.typicode.com/posts?userId=${usuarioId}`
17         const res = await fetch(url)
18         if (!res.ok) throw new Error('Error ' + res.status)
19         const datos = await res.json()
20         if (!cancelado) setPosts(datos.slice(0, 5))
21       } catch (err) {
22         if (!cancelado) setError(err.message)
23       } finally {
24         if (!cancelado) setCargando(false)
25       }
26     }
27
28     cargar()
29     return () => {
30       cancelado = true
31     }
32   }, [usuarioId]) // se repite cada vez que cambia el usuario elegido

```

```

33
34 return (
35   <div className="caja">
36     <select value={usuarioId} onChange={(e) => setUsuarioId(Number(e.target.value))}>
37       {[1, 2, 3, 4, 5].map((id) => (
38         <option key={id} value={id}>
39           Usuario {id}
40         </option>
41       ))}
42     </select>
43
44     {cargando && <p>Cargando...</p>}
45     {error && <p className="alerta">{error}</p>}
46     {!cargando && !error && (
47       <ol>
48         {posts.map((p) => (
49           <li key={p.id}>{p.title}</li>
50         ))}
51       </ol>
52     )}
53   </div>
54 )
55 }
56
57 export default PostsPorUsuario

```

Qué haces en el efecto	Qué debe limpiar el return
setInterval / setTimeout	clearInterval / clearTimeout
addEventListener	removeEventListener
Petición con fetch	Ignorar la respuesta (cancelado = true)
Cambiar document.title	Opcional: restaurar el título

18 Cuándo se vuelve a renderizar un componente

Con estado, eventos y efectos ya conoces todo lo necesario para entender las actualizaciones. Un componente se vuelve a renderizar en estos casos:

1. **Cambia su propio estado.** Al llamar a `setAlgo` con un valor distinto. Si el valor nuevo es igual al anterior, React omite el render.
2. **Se renderiza su padre.** Todos los hijos se ejecutan de nuevo, aunque sus props sean las mismas.
3. **Cambia un contexto que usa** (`useContext`, tema posterior).

Recibir props nuevas es un caso particular del punto 2: las props solo cambian porque el padre se renderizó. Modificar una variable común (`let x`) no provoca ningún render, y por eso la pantalla no se actualiza.

18.1 Experimento: pestaña “Renderizado”

El archivo `src/components/render/DemoRender.jsx` del proyecto muestra la hora exacta en que se ejecutó cada componente. Resumido:

```

1 import { useState, memo } from 'react'
2
3 function HijoConProp({ valor }) { /* muestra valor y la hora */ }
4 function HijoSinProp() { /* solo muestra la hora */ }
5 const HijoMemo = memo(function HijoMemo() { /* solo la hora */ })
6
7 function DemoRender() {
8   const [cuenta, setCuenta] = useState(0)
9   return (
10     <div>
11       <button onClick={() => setCuenta(cuenta + 1)}>Cambiar ({cuenta})</button>
12       <HijoConProp valor={cuenta} />
13       <HijoSinProp />
14       <HijoMemo />
15     </div>
16   )
17 }

```

Al presionar el botón ocurre lo siguiente:

Componente	¿Se renderiza?	Motivo
DemoRender	Sí	Cambió su estado.
HijoConProp	Sí	Su padre se renderizó (y además su prop cambió).
HijoSinProp	Sí	Su padre se renderizó, aunque no reciba nada.
HijoMemo	No	memo compara las props; como no cambiaron, reutiliza el resultado anterior.

Nota

Que un hijo se renderice de nuevo es normal y casi siempre es rápido, porque el commit solo toca lo que cambió. Usa memo cuando midas un problema real de rendimiento, no por defecto.

18.2 El estado es una foto de cada render

Dentro de un render, la variable de estado tiene un valor fijo. Llamar a `set` no la cambia de inmediato: programa un render nuevo.

```

1 const sumarTres = () => {
2   setCuenta(cuenta + 1) // cuenta vale 0 -> programa 1
3   setCuenta(cuenta + 1) // cuenta sigue valiendo 0 -> programa 1
4   setCuenta(cuenta + 1) // resultado final: 1
5   console.log(cuenta) // imprime 0
6 }
7
8 const sumarTresBien = () => {
9   setCuenta((c) => c + 1) // 0 -> 1
10  setCuenta((c) => c + 1) // 1 -> 2
11  setCuenta((c) => c + 1) // resultado final: 3
12 }

```

React además agrupa todas las llamadas a `set` de un mismo evento y hace un solo render al final. A esto se le llama *batching*.

Ejercicio de render

En la pestaña “Renderizado”, envuelve HijoSinProp en memo y comprueba que deja de actualizar su hora. Luego pásale al HijoMemo la prop valor={cuenta} y explica por qué vuelve a renderizarse.

19 Proyecto integrador: lista de tareas

Este ejemplo reúne estado, eventos, listas, formularios y renderizado condicional. Divide la interfaz en tres componentes.

src/components/FormTarea.jsx

```

1 import { useState } from 'react'
2
3 function FormTarea({ onAgregar }) {
4   const [texto, setTexto] = useState('')
5
6   const enviar = (e) => {
7     e.preventDefault()
8     if (texto.trim() === '') return
9     onAgregar(texto.trim())
10    setTexto('')
11  }
12
13  return (
14    <form onSubmit={enviar}>
15      <input value={texto} onChange={(e) => setTexto(e.target.value)}
16        placeholder="Nueva tarea" />
17      <button type="submit">Agregar</button>
18    </form>
19  )
20 }
21
22 export default FormTarea

```

src/components/ItemTarea.jsx

```

1 function ItemTarea({ tarea, onAlternar, onEliminar }) {
2   return (
3     <li>
4       <input type="checkbox" checked={tarea.hecha}
5         onChange={() => onAlternar(tarea.id)} />
6       <span style={{ textDecoration: tarea.hecha ? 'line-through' : 'none' }}>
7         {tarea.texto}
8       </span>
9       <button onClick={() => onEliminar(tarea.id)}>Eliminar</button>
10     </li>
11   )
12 }
13
14 export default ItemTarea

```

src/App.jsx

```

1 import { useState } from 'react'
2 import FormTarea from './components/FormTarea'

```

```

3 import ItemTarea from './components/ItemTarea'
4
5 function App() {
6   const [tareas, setTareas] = useState([])
7
8   const agregar = (texto) => {
9     const nueva = { id: crypto.randomUUID(), texto, hecha: false }
10    setTareas([...tareas, nueva])
11  }
12
13  const alternar = (id) => {
14    setTareas(tareas.map((t) => (t.id === id ? { ...t, hecha: !t.hecha } : t)))
15  }
16
17  const eliminar = (id) => {
18    setTareas(tareas.filter((t) => t.id !== id))
19  }
20
21  const pendientes = tareas.filter((t) => !t.hecha).length
22
23  return (
24    <main>
25      <h1>Mis tareas</h1>
26      <FormTarea onAgregar={agregar} />
27
28      {tareas.length === 0 ? (
29        <p>No hay tareas todavía.</p>
30      ) : (
31        <>
32          <ul>
33            {tareas.map((t) => (
34              <ItemTarea key={t.id} tarea={t}
35                onAlternar={alternar} onEliminar={eliminar} />
36            ))}
37          </ul>
38          <p>Pendientes: {pendientes}</p>
39        </>
40      )}
41    </main>
42  )
43 }
44
45 export default App

```

En el proyecto de acompañamiento este código está en `src/components/tareas/ListaTareas.jsx`, porque `App` se usa para la navegación. Allí las cuatro operaciones sobre la lista están extraídas a `src/logica/tareas.js`, para poder probarlas sin navegador (sección 20).

Idea clave: el estado vive en `App`, que es el componente padre. Los hijos no modifican la lista, solo avisan al padre llamando a las funciones que reciben por props. Este patrón se llama *levantar el estado*.

Ejercicio 1

Agrega tres botones de filtro (*Todas*, *Pendientes*, *Completadas*). Guarda el filtro activo en un nuevo estado y aplica `filter` antes del `map`.

Ejercicio 2

Haz que las tareas no se pierdan al recargar la página. Usa un `useEffect` con `[tareas]` como dependencia para guardar en `localStorage`, e inicializa el estado leyendo desde ahí: `useState(() => JSON.parse(localStorage.getItem('tareas')) ?? [])`.

Ejercicio 3

Agrega un botón “Eliminar completadas” que solo aparezca cuando exista al menos una tarea hecha.

20 Pruebas unitarias con Jasmine y Karma

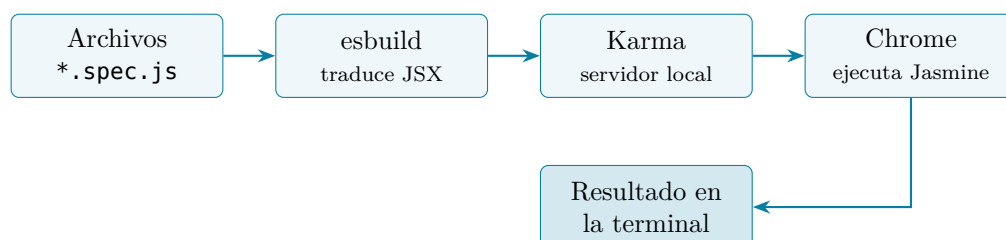
Una **prueba unitaria** es un trozo de código que comprueba automáticamente que otro trozo de código hace lo que debe. En vez de abrir el navegador y revisar a mano que la nota 3,9 diga “Reprobado”, escribes una vez esa comprobación y la ejecutas cuantas veces quieras. Cuando el proyecto crece, es la forma de saber en segundos si un cambio rompió algo que antes funcionaba.

Tipo	Qué comprueba
Unitaria	Una función o un componente aislado. Son rápidas y numerosas.
Integración	Varias piezas trabajando juntas, por ejemplo un formulario con su validación.
Extremo a extremo	La aplicación completa en un navegador, como la usaría una persona.

Esta sección cubre las unitarias, que son las primeras que conviene aprender.

20.1 Qué hace cada herramienta

Paquete	Rol
jasmine-core	El marco de pruebas. Aporta <code>describe</code> , <code>it</code> , <code>expect</code> , los comparadores y los espías. Es quien define el lenguaje con que escribes las pruebas.
karma	El ejecutor. Levanta un servidor, abre un navegador de verdad, carga ahí tus pruebas y muestra los resultados en la terminal.
karma-jasmine	Une a los dos anteriores.
karma-chrome-launcher	Permite que Karma abra y cierre Chrome.
karma-esbuild	Traduce el JSX y resuelve los <code>import</code> antes de enviar el código al navegador.



Atención

El equipo de Karma anunció en 2023 que el proyecto está descontinuado: sigue funcionando, pero ya no recibe nuevas características. Muchos cursos y proyectos en producción lo siguen usando con Jasmine, y lo que aprendas aquí se traslada casi sin cambios a herramientas actuales como Vitest (sección [20.15](#)).

20.2 Instalación y configuración

```
npm install -D karma karma-jasmine karma-chrome-launcher karma-esbuild jasmine-core
```

Necesitas Chrome instalado en el equipo. Agrega los scripts a `package.json`:

```
"scripts": {  
  "test": "karma start",  
  "test:watch": "karma start --no-single-run --auto-watch"  
}
```

Crea el archivo de configuración en la raíz del proyecto:

karma.conf.cjs

```
// Configuración de Karma. Usa .cjs porque package.json tiene "type": "module".  
module.exports = function (config) {  
  config.set({  
    // Carpeta base desde donde se resuelven las rutas  
    basePath: '',  
  
    // Jasmine aporta describe, it, expect y los espías  
    frameworks: ['jasmine'],  
  
    // Archivos que Karma carga en el navegador  
    files: [{ pattern: 'src/**/*.spec.js*', watched: false, type: 'module' }],  
  
    // esbuild traduce JSX y resuelve los import antes de enviarlos al navegador  
    preprocessors: { 'src/**/*.spec.js*': ['esbuild'] },  
  
    esbuild: {  
      target: 'es2022',  
      jsx: 'automatic', // permite escribir JSX sin importar React  
      loader: { '.js': 'jsx' },  
    },  
  
    // Cómo se muestran los resultados en la terminal  
    reporters: ['progress'],  
  
    // Navegador donde corren las pruebas. Chrome muestra la ventana;  
    // ChromeHeadless corre sin interfaz, útil en servidores.  
    browsers: ['ChromeHeadless'],  
  
    // true: corre una vez y termina. false: queda esperando cambios.  
    singleRun: true,  
    autoWatch: false,  
  
    port: 9876,  
    colors: true,  
    logLevel: config.LOG_INFO,  
    concurrency: 1,  
  });  
}
```

```

37   })
38 }

```

Lo importante de ese archivo:

frameworks: ['jasmine']

Carga Jasmine antes de tus pruebas, por eso `describe` e `it` existen sin importarlos.

files y preprocessors

El patrón `src/**/*.spec.js*` toma todos los archivos que terminen en `.spec.js` o `.spec.jsx`, en cualquier subcarpeta de `src`. Cada uno pasa primero por `esbuild`.

esbuild.jsx: 'automatic'

Permite escribir JSX en las pruebas de componentes sin importar React.

browsers

ChromeHeadless corre Chrome sin abrir ventana. Usa Chrome si quieres ver el navegador y depurar con F12.

singleRun

En `true` corre una vez y termina, que es lo que se usa al entregar o en un servidor de integración continua.

Nota

El nombre del archivo termina en `.cjs` porque `package.json` tiene `"type": "module"`. Con esa opción, todos los `.js` del proyecto se tratan como módulos ES, y Karma espera este archivo en el formato antiguo de Node.

20.3 Anatomía de una prueba

```

1 describe('nombre del grupo', () => { // agrupa pruebas relacionadas
2   it('describe una conducta esperada', () => {
3     const resultado = funcionQueSePrueba(entrada) // 1. preparar y ejecutar
4     expect(resultado).toBe(valorEsperado)          // 2. comprobar
5   })
6 })

```

- `describe` agrupa. Se pueden anidar para organizar por función o por caso.
- `it` es una prueba. Su texto completa la frase “it...” (“debería...”), así que se escribe como una conducta: “*suma uno por cada clic*”, no “*prueba 1*”.
- `expect(...)` toma el valor obtenido y se compara con un **comparador** (*matcher*).

Una forma útil de ordenar cada prueba es el patrón **preparar, ejecutar, comprobar**: primero los datos de entrada, después la llamada, al final las comprobaciones.

20.4 Primer ejemplo

`src/logica/formato.js`

```

1 // Funciones puras: misma entrada, misma salida, sin tocar el DOM.
2 // Al estar separadas de los componentes se pueden probar con facilidad.

```

```

3
4 export function formatearPesos(valor) {
5   return '$' + valor.toLocaleString('es-CL')
6 }
7
8 export function numeroAleatorio(maximo = 100) {
9   return Math.floor(Math.random() * maximo) + 1
10 }

```

src/logica/formato.spec.js

```

1 import { formatearPesos, numeroAleatorio } from './formato'
2
3 // describe agrupa pruebas relacionadas
4 describe('formatearPesos', () => {
5   // it describe una conducta esperada, en una frase
6   it('agrega el signo peso y separa los miles', () => {
7     expect(formatearPesos(25990)).toBe('$25.990')
8   })
9
10  it('funciona con el cero', () => {
11    expect(formatearPesos(0)).toBe('$0')
12  })
13
14  it('devuelve un texto', () => {
15    expect(typeof formatearPesos(100)).toBe('string')
16  })
17 })
18
19 describe('numeroAleatorio', () => {
20   it('entrega un valor dentro del rango pedido', () => {
21     for (let i = 0; i < 50; i++) {
22       const n = numeroAleatorio(10)
23       expect(n).toBeGreaterThanOrEqual(1)
24       expect(n).toBeLessThanOrEqual(10)
25     }
26   })
27
28   it('usa Math.random (espía)', () => {
29     // El espía reemplaza a Math.random y siempre devuelve 0.5
30     spyOn(Math, 'random').and.returnValue(0.5)
31     expect(numeroAleatorio(100)).toBe(51)
32     expect(Math.random).toHaveBeenCalled()
33     expect(Math.random).toHaveBeenCalledTimes(1)
34   })
35 })

```

El archivo de pruebas vive junto al que prueba y repite su nombre con `.spec` al medio. Así siempre se sabe qué está probado y qué no.

20.5 Comparadores de Jasmine

Comparador	Comprueba que el valor
<code>toBe(x)</code>	Es idéntico a x (comparación <code>===</code>). Para números, textos y booleanos.
<code>toEqual(x)</code>	Tiene el mismo contenido que x. Para objetos y arreglos.
<code>toBeTrue()</code> / <code>toBeFalse()</code>	Es exactamente <code>true</code> o <code>false</code> .
<code>toBeTruthy()</code> / <code>toBeFalsy()</code>	Se evalúa como verdadero o falso.
<code>toBeDefined()</code> / <code>toBeUndefined()</code>	Existe o no existe.
<code>toBeNull()</code>	Es <code>null</code> .
<code>toContain(x)</code>	Contiene x, en un arreglo o en un texto.
<code>toBeGreaterThan(n)</code>	Es mayor que n. También existen <code>toBeLessThan</code> y las versiones <code>OrEqual</code> .
<code>toBeCloseTo(n, d)</code>	Se acerca a n con d decimales. Para cálculos con decimales.
<code>toThrowError(msg)</code>	Lanza un error con ese mensaje. Se usa con una función: <code>expect(() => f()).toThrowError()</code> .
<code>toHaveBeenCalled()</code>	Fue llamado. Solo para espías.

Cualquier comparador se puede negar con `not`: `expect(x).not.toBe(3)`.

Atención

`toBe` y `toEqual` no son lo mismo. `expect([1, 2]).toBe([1, 2])` falla, porque son dos arreglos distintos en memoria aunque tengan el mismo contenido. Con objetos y arreglos usa siempre `toEqual`.

20.6 Preparar datos con `beforeEach`

Cuando varias pruebas necesitan los mismos datos, se preparan en `beforeEach`, que se ejecuta antes de cada `it`. Así ninguna prueba hereda los cambios de la anterior.

src/logica/cursos.spec.js

```
import { filtrarPorNombre, ordenarCursos } from './cursos'

describe('cursos', () => {
  let cursos

  // beforeEach corre antes de cada it: cada prueba parte con datos limpios
  beforeEach(() => {
    cursos = [
      { id: 1, nombre: 'Programación Web', creditos: 8 },
      { id: 2, nombre: 'Arquitectura de Software', creditos: 6 },
      { id: 3, nombre: 'Desarrollo Fullstack', creditos: 12 },
    ]
  })

  describe('filtrarPorNombre', () => {
    it('encuentra sin distinguir mayúsculas', () => {
      expect(filtrarPorNombre(cursos, 'DESARROLLO').length).toBe(1)
    })

    it('devuelve todos si la búsqueda está vacía', () => {
      expect(filtrarPorNombre(cursos, ' ').length).toBe(3)
    })

    it('devuelve un arreglo vacío si no hay coincidencias', () => {
```

```

25     expect(filtrarPorNombre(cursos, 'química')).toEqual([])
26   })
27 })
28
29 describe('ordenarCursos', () => {
30   it('ordena alfabéticamente por nombre', () => {
31     const nombres = ordenarCursos(cursos, 'nombre').map((c) => c.nombre)
32     expect(nombres).toEqual(['Arquitectura de Software', 'Desarrollo Fullstack', '
33     Programación Web'])
34   })
35
36   it('ordena por créditos de mayor a menor', () => {
37     expect(ordenarCursos(cursos, 'creditos')[0].creditos).toBe(12)
38   })
39
40   it('no modifica el arreglo original', () => {
41     ordenarCursos(cursos, 'nombre')
42     expect(cursos[0].nombre).toBe('Programación Web')
43   })
44 })

```

Jasmine ofrece cuatro ganchos: `beforeEach` y `afterEach` (antes y después de cada prueba) y `beforeAll` y `afterAll` (una vez por grupo).

20.7 Probar los casos límite

Los errores se esconden en los bordes: el cero, el arreglo vacío, el texto en blanco, el valor justo en el límite. En una escala de 1 a 7 con 4 como nota mínima, hay que probar 3,9 y 4,0.

```

src/logica/notas.spec.js
1 import { estaAprobado, mensajePara, NOTA_MINIMA } from './notas'
2
3 describe('estaAprobado', () => {
4   it('aprueba desde la nota mínima', () => {
5     expect(estaAprobado(NOTA_MINIMA)).toBeTrue()
6     expect(estaAprobado(4.0)).toBeTrue()
7   })
8
9   it('reprueba bajo la nota mínima', () => {
10    expect(estaAprobado(3.9)).toBeFalse()
11  })
12 })
13
14 describe('mensajePara', () => {
15   // Los casos límite son los que más errores descubren
16   const casos = [
17     { nota: 7.0, esperado: 'Excelente' },
18     { nota: 6.0, esperado: 'Excelente' },
19     { nota: 5.9, esperado: 'Aprobado' },
20     { nota: 4.0, esperado: 'Aprobado' },
21     { nota: 3.9, esperado: 'Reprobado' },
22     { nota: 1.0, esperado: 'Reprobado' },
23   ]
24
25   casos.forEach(({ nota, esperado }) => {
26     it(`con nota ${nota} devuelve "${esperado}"`, () => {

```

```

27     expect(mensajePara(nota)).toBe(esperado)
28   })
29 })
30 })

```

Como `it` es una función normal, se puede llamar dentro de un `forEach` para generar una prueba por caso sin repetir código.

20.8 Separar la lógica de la interfaz

Una función que solo recibe datos y devuelve datos se prueba en una línea. Un componente, en cambio, necesita un navegador, un contenedor y simulación de clics. Por eso conviene sacar las reglas del negocio del componente y dejarlas en funciones puras.

En el proyecto de acompañamiento, la carpeta `src/logica/` contiene todo lo que se puede probar sin navegador:

Archivo	Qué contiene
<code>formato.js</code>	Formato de precios y número aleatorio.
<code>seguridad.js</code>	Nivel de seguridad de una contraseña.
<code>notas.js</code>	Aprobación y mensaje según la nota.
<code> cursos.js</code>	Filtrar y ordenar cursos.
<code>inscripcion.js</code>	Validación del formulario.
<code>tareas.js</code>	Crear, agregar, alternar, eliminar y contar tareas.
<code>api.js</code>	Obtener títulos desde la API.

El componente queda más corto y se limita a mostrar:

```

1 // Antes: la regla estaba escrita dentro del componente
2 let seguridad = 'Debil'
3 if (clave.length >= 8 && /\d/.test(clave)) seguridad = 'Fuerte'
4 else if (clave.length >= 6) seguridad = 'Media'
5
6 // Despues: la regla vive en src/logica/seguridad.js y tiene sus pruebas
7 const seguridad = nivelSeguridad(clave)

```

20.9 Espías

Un **espía** (*spy*) reemplaza una función para observar cómo se usó o para controlar lo que devuelve. Sirve cuando el código depende de algo impredecible (el azar, la hora) o costoso (la red).

```

1 spyOn(Math, 'random').and.returnValue(0.5) // reemplaza y fija el resultado
2 spyOn(objeto, 'metodo').and.callThrough()   // observa sin cambiar la conducta
3 spyOn(objeto, 'metodo').and.callFake(fn)    // lo reemplaza por otra funcion
4 const espia = jasmine.createSpy('nombre')  // una funcion falsa desde cero
5
6 expect(espia).toHaveBeenCalled()
7 expect(espia).toHaveBeenCalledTimes(2)
8 expect(espia).toHaveBeenCalledWith('Leer')
9 espia.calls.mostRecent().args // argumentos de la ultima llamada

```

Los espías creados con `spyOn` se deshacen solos al terminar cada prueba.

20.10 Pruebas asíncronas

Para probar código que espera una respuesta, la función del `it` se declara `async` y se usa `await`. En lugar de salir a internet, se le entrega a la función una versión falsa de `fetch`: la prueba queda rápida y su resultado no depende de la conexión.

src/logica/api.js

```
// El segundo parámetro permite reemplazar fetch por una función falsa en las pruebas.
export async function obtenerTitulos(usuarioId, traer = fetch) {
  const res = await traer(`https://jsonplaceholder.typicode.com/posts?userId=${usuarioId}`)
  if (!res.ok) throw new Error('Error ' + res.status)
  const datos = await res.json()
  return datos.map((p) => p.title)
}
```

src/logica/api.spec.js

```
import { obtenerTitulos } from './api'

describe('obtenerTitulos', () => {
  it('devuelve solo los títulos', async () => {
    // Doble de prueba: imita la respuesta de fetch sin salir a internet
    const traerFalso = jasmine.createSpy('traer').and.returnValue(
      Promise.resolve({
        ok: true,
        json: () => Promise.resolve([
          { title: 'uno' },
          { title: 'dos' }
        ]),
      })
    )

    const titulos = await obtenerTitulos(3, traerFalso)

    expect(titulos).toEqual(['uno', 'dos'])
    expect(traerFalso).toHaveBeenCalledTimes(1)
    // Se revisa que la URL incluya el usuario pedido
    expect(traerFalso.calls.mostRecent().args[0]).toContain('userId=3')
  })

  it('lanza un error si la respuesta falla', async () => {
    const traerFalso = () => Promise.resolve({ ok: false, status: 404 })
    await expectAsync(obtenerTitulos(1, traerFalso)).toBeRejectedWithError('Error 404')
  })
})
```

`expectAsync` es la versión de `expect` para promesas. Sus comparadores son `toBeResolved`, `toBeResolvedTo`, `toBeRejected` y `toBeRejectedWithError`.

20.11 Probar un componente

Para probar un componente hay que montarlo en un contenedor, provocar la interacción y revisar lo que quedó en pantalla. `act` agrupa cada acción y espera a que React termine de renderizar antes de continuar.

src/components/basicos/Contador.spec.jsx

```
import { act } from 'react'
import { createRoot } from 'react-dom/client'
import Contador from './Contador'
```

```

4
5 // React necesita saber que está en un entorno de pruebas
6 globalThis.IS_REACT_ACT_ENVIRONMENT = true
7
8 describe('Contador (componente)', () => {
9   let contenedor
10   let raiz
11
12   beforeEach(() => {
13     contenedor = document.createElement('div')
14     document.body.appendChild(contenedor)
15     raiz = createRoot(contenedor)
16   })
17
18   afterEach(() => {
19     // Limpieza: cada prueba debe partir con la pantalla vacía
20     act(() => raiz.unmount())
21     contenedor.remove()
22   })
23
24   const renderizar = () => act(() => raiz.render(<Contador />))
25   const clicEn = (boton) => act(() => boton.dispatchEvent(new MouseEvent('click', {
26     bubbles: true })))
27
28   it('parte en cero', () => {
29     renderizar()
30     expect(contenedor.textContent).toContain('Has hecho clic 0 veces')
31   })
32
33   it('suma uno por cada clic', () => {
34     renderizar()
35     const sumar = contenedor.querySelectorAll('button')[0]
36     clicEn(sumar)
37     clicEn(sumar)
38     expect(contenedor.textContent).toContain('Has hecho clic 2 veces')
39   })
40
41   it('vuelve a cero con el botón Reiniciar', () => {
42     renderizar()
43     const [sumar, reiniciar] = contenedor.querySelectorAll('button')
44     clicEn(sumar)
45     clicEn(reiniciar)
46     expect(contenedor.textContent).toContain('Has hecho clic 0 veces')
47   })
48 })

```

IS_REACT_ACT_ENVIRONMENT

Le indica a React que está en pruebas. Sin esta línea aparece una advertencia en consola.

beforeEach y afterEach

Crean y destruyen el contenedor. Sin la limpieza, los componentes de una prueba quedarían en la página durante la siguiente.

contenedor.textContent

Es el texto visible. Se comprueba lo que ve el usuario, no cómo está implementado el componente por dentro.

dispatchEvent

Simula el clic. La opción `bubbles: true` es necesaria porque React escucha los eventos en la raíz.

Nota

La biblioteca **React Testing Library** hace esto mismo con menos código (`render`, `screen.getByText`, `fireEvent.click`) y es el estándar actual para probar componentes. Vale la pena mirarla después de entender lo que ocurre por debajo.

20.12 Ejecutar las pruebas

```
npm test          # corre todo una vez y muestra el resumen
npm run test:watch # queda escuchando y repite al guardar
```

La salida indica cuántas pruebas pasaron y detalla las que fallaron con el valor esperado y el obtenido:

```
Chrome Headless 140.0.0: Executed 44 of 44 SUCCESS (0.35 secs / 0.29 secs)
```

Mientras escribes una prueba puedes enfocarte en una sola: `fit` y `fdescribe` ejecutan únicamente esa prueba o ese grupo; `xit` y `xdescribe` los saltan. Recuerda devolverlos a `it` y `describe` antes de entregar.

Atención

Si aparece *No binary for ChromeHeadless browser*, Karma no encontró Chrome. Indícale la ruta con la variable `CHROME_BIN`:

Windows (PowerShell):

```
$env:CHROME_BIN="C:\Program Files\Google\Chrome\Application\chrome.exe"
```

Linux: `export CHROME_BIN=$(which google-chrome)`

20.13 Errores frecuentes al probar

Síntoma	Causa y solución
<i>Expected [1, 2] to be [1, 2]</i>	Usaste <code>toBe</code> con un arreglo u objeto. Cambia a <code>toEqual</code> .
La prueba pasa sola pero falla junto a otras	Alguna prueba modifica datos compartidos. Crea los datos dentro de <code>beforeEach</code> .
<i>Executed 0 of 0</i>	Ningún archivo coincide con el patrón de <code>files</code> . Revisa el nombre <code>.spec.js</code> .
La prueba asíncrona pasa siempre	Olvidaste <code>await</code> o marcar la función como <code>async</code> .
<i>An update to X inside a test was not wrapped in act</i>	Falta envolver la interacción en <code>act(...)</code> .
La prueba se rompe al cambiar el diseño	Estás comprobando detalles internos. Compara el texto visible.

20.14 Buenas prácticas

- Una conducta por prueba. Si el nombre necesita un “y”, probablemente son dos pruebas.
- Cada prueba debe poder correr sola y en cualquier orden.

- Prueba lo que hace tu código, no lo que hace React ni otras bibliotecas.
- Cuando aparezca un error, escribe primero una prueba que lo reproduzca y después corrígelo. Esa prueba evita que el error vuelva.
- Mide la cobertura si quieres, pero recuerda que un 100% no garantiza que las comprobaciones sean buenas.

20.15 La alternativa actual: Vitest

Vitest usa la misma configuración de Vite, no necesita navegador y su sintaxis es casi idéntica:

```
npm install -D vitest jsdom @testing-library/react
```

```
1 import { describe, it, expect } from 'vitest' // en Jasmine son globales
2 import { mensajePara } from './notas'
3
4 describe('mensajePara', () => {
5   it('aprueba con 4.0', () => {
6     expect(mensajePara(4.0)).toBe('Aprobado') // identico a Jasmine
7   })
8 })
```

Los cambios principales son que las funciones se importan, que los espías se crean con `vi.fn()` y `vi.spyOn()`, y que se ejecuta con `npx vitest`. Todo lo aprendido sobre qué probar sigue igual.

Ejercicio de pruebas 1

Escribe `src/logica/carrito.js` con las funciones `agregarProducto`, `quitarProducto` y `totalCarrito`, y su archivo de pruebas. Incluye el caso del carrito vacío.

Ejercicio de pruebas 2

Rompe a propósito `nivelSeguridad`, cambiando el 8 por un 4, y ejecuta `npm test`. Observa qué prueba falla y qué mensaje entrega. Esa es la utilidad real de tener pruebas.

Ejercicio de pruebas 3

Escribe las pruebas del componente `Interruptor`: que el texto esté oculto al inicio, que aparezca con el primer clic y que desaparezca con el segundo.

21 Generar la versión de producción

Cuando la aplicación esté lista:

```
npm run build
npm run preview
```

`build` crea la carpeta `dist/` con archivos HTML, CSS y JavaScript optimizados. Ese contenido se puede publicar gratis en servicios como Netlify, Vercel o GitHub Pages.

Nota

Antes de subir el proyecto a GitHub, verifica que `.gitignore` incluya `node_modules` y `dist`. Vite ya lo trae configurado. Quien clone el repositorio solo debe ejecutar `npm install`.

22 Errores frecuentes

Síntoma	Causa y solución
<i>Adjacent JSX elements must be wrapped</i>	Devuelves varios elementos sin padre. Envuélvelos en <code><>...</></code> .
El componente no aparece o sale como etiqueta HTML	El nombre empieza con minúscula. Usa <code>Tarjeta</code> , no <code>tarjeta</code> .
<i>Each child in a list should have a unique key</i>	Falta <code>key</code> en los elementos del <code>map</code> .
<i>Too many re-renders</i>	Llamas una función en vez de pasarla: <code>onClick={f()}</code> . Cambia a <code>onClick={f}</code> o <code>onClick={() => f(x)}</code> .
La pantalla no cambia al modificar datos	Modificaste el estado directamente (<code>push</code> , asignación). Crea un arreglo u objeto nuevo y usa <code>set</code> .
<i>Failed to resolve import</i>	Ruta o nombre de archivo mal escrito. Revisa mayúsculas y la extensión.
<i>Port 5173 is in use</i>	Hay otro servidor corriendo. Ciérralo o acepta el puerto alternativo que propone Vite.
El input no deja escribir	Tiene <code>value</code> pero no <code>onChange</code> .

23 Resumen de comandos

```
node -v           # version de Node
npm -v           # version de npm
npm create vite@latest app -- --template react
cd app
npm install       # instala dependencias
npm run dev       # servidor de desarrollo
npm install nombre-paquete # agrega una libreria
npm test          # pruebas unitarias
npm run build     # version de produccion
npm run preview   # prueba la version final
```

24 Glosario

Término	Significado
Componente	Función que devuelve JSX. Su nombre empieza con mayúscula.
JSX	Sintaxis parecida a HTML que se escribe dentro de JavaScript.
Props	Datos que un componente recibe de su padre. Son de solo lectura.
children	Prop especial con el contenido escrito entre las etiquetas del componente.
Estado	Datos que un componente recuerda y que, al cambiar, provocan un nuevo render.
Valor derivado	Dato que se calcula a partir del estado o de las props en cada render.
Hook	Función de React que empieza con <code>use</code> (<code>useState</code> , <code>useEffect</code>).
Render	Ejecución de la función del componente para obtener su JSX.
Commit	Fase en que React aplica al DOM las diferencias encontradas.
Montar / desmontar	Aparecer en pantalla por primera vez / ser eliminado de ella.
Efecto	Código que sincroniza el componente con algo externo a React.
DOM	Árbol de objetos que el navegador construye a partir del HTML.
SPA	Aplicación de una sola página cuyo contenido cambia con JavaScript.
key	Identificador único de cada elemento de una lista.
Input controlado	Campo cuyo valor está guardado en el estado.
Fragmento	<code><>...</></code> : agrupa elementos sin agregar etiquetas al DOM.
Paquete	Biblioteca instalada con npm. Se registra en <code>package.json</code> .
Levantar el estado	Mover el estado al padre común de los componentes que lo necesitan.

25 Próximos pasos

Con lo visto ya puedes construir aplicaciones de una sola vista. El orden sugerido para seguir avanzando es:

1. **React Router**: navegación entre páginas (`npm install react-router`).
2. **Hooks adicionales**: `useRef`, `useContext` para compartir datos sin pasar props por muchos niveles, y `useReducer` para estados complejos.
3. **Hooks personalizados**: extraer lógica repetida, por ejemplo un `useFetch`.
4. **Estilos**: Tailwind CSS o CSS Modules.
5. **TypeScript**: tipar props y estado. Se crea con la plantilla `react-ts`.
6. **Pruebas de componentes**: React Testing Library, sobre lo visto en la sección 20.
7. **Frameworks**: Next.js o React Router en modo framework, para renderizado en servidor.

Documentación oficial: <https://react.dev/learn>. Está disponible en español en <https://es.react.dev> e incluye ejercicios interactivos al final de cada tema.